



Debre Berhan University

College of Computing

Department of Computer Science

Intelligent Systems Module



Part I: Introduction to Artificial Intelligence

*March 2023
Debre Berhan,
Ethiopia*

Table of Contents

Chapter One: Introduction to artificial intelligence.....	4
1.1 Definition.....	4
1.2 Typical AI problems.....	6
1.3 Intelligent behaviour.....	6
1.4 Practical Impact of AI	6
1.5 Approaches to AI	6
1.6 Limits of AI Today.....	7
1.7 What can AI systems do?.....	7
1.8 What can AI systems NOT do yet?.....	7
1.9 Goals of AI	8
1.10 AI Techniques	8
1.11 Sample questions	8
Chapter Two: Intelligent Agents	3
2.1 Introduction to Intelligent Agents.....	3
2.2 Agents and the environments	4
2.3 Rationality Vs Omniscience	4
2.4 Structure of Intelligent Agents	5
2.5 Autonomy	5
2.6 Task Environments	6
2.7 Properties of Task environments	6
2.8 PEAS examples	6
2.9 Agent Types.....	7
2.10 Summery.....	9
2.11 Questions	9
Chapter Three: Problem Solving (Goal Based) Agents.....	2
3.1 Problem Solving by Searching	2
3.2 What is search and terminologies?	2
3.5 Search Strategies	2
3.5.1 Unformed Search Strategies	2
3.5.2 Informed Search Strategies.....	8
3.5.3 Local Search Strategies	12
3.5.4 Adversarial Search Strategies.....	16
3.6 Avoiding Repeated States.....	19
3.7 Constraint Satisfaction Search.....	20
3.8 Sample Questions	22
Chapter Four: Knowledge Representation and Reasoning.....	23
4.1 Knowledge based agent	24

4.2	Architecture of Knowledge based agents	24
4.3	Levels of knowledge.....	25
4.4	Approaches to design KB agents.....	26
4.5	Knowledge Representation.....	26
4.6	Techniques of Knowledge Representation	26
4.7	Propositional Logic	29
4.7.1	Logical connectives in PL	30
4.7.2	Inference in Propositional Logic	1
4.8	Predicate (First-Order) Logic	2
4.8.1	Quantifiers in FOL.....	3
4.8.2	Inference in First-Order Logic.....	4
4.8.3	Unification in FOL	6
4.8.4	Resolution in FOL	7
4.9	Reasoning	7
4.10	Reasoning under uncertainty	9
4.11	Summery.....	12
4.12	Sample Questions	12
	Chapter Five: Expert System.....	2
5.1	Introduction	2
5.2	Applications of Expert Systems	2
5.3	Expert Systems Technologies.....	2
5.4	Benefits of Expert Systems.....	2
5.5	Expert System Limitations	3
5.6	The Architecture of Expert Systems.....	3
5.7	Components of Expert Systems.....	3
5.7.1	The Knowledge bases.....	3
5.7.2	The Inference Engine.....	4
5.7.3	The User Interface	5
5.8	Development of Expert System.....	6
5.7	Questions	6
	Chapter Six: Learning Agents	2
6.1	Introduction	2
6.2	Types of learning (machine learning).....	3
A.	Decision tree algorithm	4
B.	Regression algorithm.....	5
6.2	Neural Networks.....	6
6.3	Biological neurons.....	7
6.4	ANN for linear equations	8

6.5	Processing of ANN.....	9
6.6	Application of ANN	11
6.7	Sample Questions	12
Chapter Seven: Communicating, Perceiving, and Acting		1
7.1	Natural language processing.....	2
7.2	Applications of NLP	4
7.3	Introduction to robotics	5
7.4	Sample Questions	7

Chapter One: Introduction to artificial intelligence

Objectives

After completing this chapter, students will be able to know and understand

- Definition of Artificial Intelligence
- Typical AI problems
- Intelligent behaviour
- Practical Impact of AI
- Approaches to AI
- Limits of AI Today
- What can AI systems do?
- What can AI systems NOT do yet?
- Goals of AI
- General AI goal
- AI Techniques

1.1 Definition

AI is the study of the mental faculties through the use of computational models. Artificial Intelligence is concerned with the design of intelligence in an artificial device. The term was coined by McCarthy in 1956. There are two ideas in the definition. i.e Intelligence, and Artificial device.

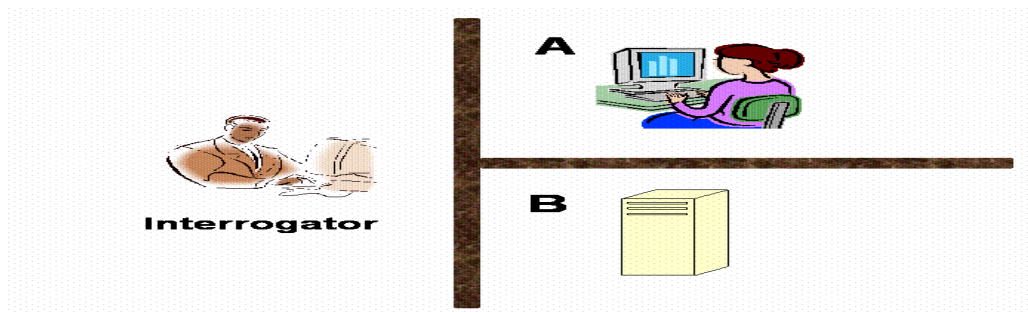
What is intelligence?

- Intelligence relate to tasks involving higher mental process e.g. creativity, solving problems, pattern recognition, classification, classification, learning, induction, deduction, building analogies, optimization language processing, knowledge and many more.
- Intelligence is the computational part of the ability to achieve goals.
- Is it that which characterize humans? Or is there an absolute standard of judgment?
- Accordingly, there are two possibilities:

- A system with intelligence is expected to behave as intelligently as a human
- A system with intelligence is expected to behave in the best possible manner
- Secondly what type of behaviour are we talking about?
 - Are we looking at the thought process or reasoning ability of the system?
 - Or are we only interested in the final manifestations of the system in terms of its actions?

Given this scenario different interpretations have been used by different researchers as defining the scope and view of Artificial Intelligence.

1. One view is that artificial intelligence is about designing systems that are as intelligent as humans. This view involves trying to understand human thought and an effort to build machines that emulate the human thought process. This view is the **cognitive** science approach to AI. –*Think human like*
2. The second approach is best embodied by the concept of the Turing Test. Turing held that in future computers can be programmed to acquire abilities rivaling human intelligence. As part of his argument Turing put forward the idea of an '**imitation game**', in which a human being and a computer would be interrogated under conditions where the interrogator would not know which was which, the communication being entirely by textual messages. Turing argued that if the interrogator could not distinguish them by questioning, then it would be unreasonable not to call the computer intelligent. Turing's 'imitation game' is now usually called 'the Turing test' for intelligence.



Turing Test

3. Logic and **laws of thought** deals with studies of ideal or rational thought process and inference. The emphasis in this case is on the inference mechanism, and its properties. That is how the system arrives at a conclusion, or the reasoning behind its selection of actions is very important in this point of view. The soundness and completeness of the inference mechanisms are important here. –*think rationally*
4. The fourth view of AI is that it is the study of **rational agents**. This view deals with building

machines that **act rationally**. The focus is on how the system acts and performs, and not so much on the reasoning process. A rational agent is one that acts rationally, that is, in the best possible manner.

1.2 Typical AI problems

While studying the typical range of tasks that we might expect an “intelligent entity” to perform, we need to consider both “common-place” tasks as well as expert tasks.

Examples of common-place tasks include

- *Recognizing* people, objects.
- Communicating (through *natural language*).
- *Navigating* around obstacles on the streets

1.3 Intelligent behaviour

This discussion brings us back to the question of what constitutes intelligent behavior. Some of these tasks and applications are:

- Perception involving image recognition and computer vision
- Reasoning
- Learning
- Understanding language involving natural language processing, speech processing
- Solving problems
- Robotics
- Thinking
- Acting- in the complex environment
- Knowledge – applying successfully in new situation

1.4 Practical Impact of AI

AI components are embedded in numerous devices e.g. in copy machines for automatic correction of operation for copy quality improvement. AI systems are in everyday use for identifying credit card fraud, for advising doctors, for recognizing speech and in helping complex planning tasks. Then there are intelligent tutoring systems that provide students with personalized attention

Thus AI has increased understanding of the nature of intelligence and found many applications. It has helped in the understanding of human reasoning, and of the nature of intelligence. It has also helped us understand the complexity of modeling human reasoning.

1.5 Approaches to AI

Strong AI aims to build machines that can truly reason and solve problems. These machines should be self-aware and their overall intellectual ability needs to be indistinguishable from that of a human being. Excessive optimism in the 1950s and 1960s concerning strong AI has given way to an appreciation of the extreme difficulty of the problem. Strong AI maintains that suitably programmed

machines are capable of cognitive mental states.

Weak AI: deals with the creation of some form of computer-based artificial intelligence that cannot truly reason and solve problems, but can act as if it were intelligent. Weak AI holds that suitably programmed machines can simulate human cognition.

Applied AI: aims to produce commercially viable smart system. For example, a security system that is able to recognize the faces of people who are permitted to enter a particular building. Applied AI has already enjoyed considerable success.

Cognitive AI: computers are used to test theories about how the human mind works--for example, theories about how we recognize faces and other objects, or about how we solve abstract problems.

1.6 Limits of AI Today

Today's successful AI systems operate in well-defined domains and employ narrow, specialized knowledge. Common sense knowledge is needed to function in complex, open-ended worlds. Such a system also needs to understand unconstrained natural language. However these capabilities are not yet fully present in today's intelligent systems.

1.7 What can AI systems do?

Today's AI systems have been able to achieve limited success in some of these tasks.

- In Computer vision, the systems are capable of face recognition
- In Robotics, we have been able to make vehicles that are mostly autonomous.
- In Natural language processing, we have systems that are capable of simple machine translation.
- Today's Expert systems can carry out medical diagnosis in a narrow domain
- Speech understanding systems are capable of recognizing several thousand words continuous speech
- Planning and scheduling systems had been employed in scheduling experiments with the Hubble Telescope
- The Learning systems are capable of doing text categorization into about a 1000 topics
- In Games, AI systems can play at the Grand Master level in chess (world champion), checkers, etc.

1.8 What can AI systems NOT do yet?

- Understand natural language robustly (e.g., read and understand articles in a newspaper)
- Surf the web
- Interpret an arbitrary visual scene
- Learn a natural language
- Construct plans in dynamic real-time domains

- Exhibit true autonomy and intelligence

1.9 Goals of AI

The definition of AI gives four possible goals to pursue

- | | |
|---------------------------------|------------------------------|
| ⇒ systems that think like human | □ cognitive science approach |
| ⇒ systems that act like human | □ Turing Test approach |
| ⇒ systems that think rationally | □ Law of thought approach |
| ⇒ systems that act rationally | □ rational agent approach |

General AI goal

- Replicate human intelligent
- Solve knowledge intensive task
- Make an intelligent connection between perception and action
- Enhance human-computer interaction/ communication

Engineering based AI goal

- Develop concepts, theory and practice of building intelligent machines
- Emphasis is on system building

Science based AI goal

- Develop concepts, mechanisms and vocabulary to understand biological intelligent behaviours
- Emphasis is on understanding intelligent behaviours

1.10 AI Techniques

Various techniques that have involved can be applied to a variety of AI tasks. The techniques are concerned with how we represent, manipulate and reason with knowledge in order to solve problem.

Example

- Techniques, not all “intelligent” but used to behave as intelligent
 - Describe and match
 - Constraint satisfaction
 - Generate and Test
 - Goal reduction
 - Tree search
 - Rule based system
- Biology-inspired AI techniques are currently popular
 - Neural Network
 - Reinforcement learning
 - Genetic Algorithms

1.11 Sample questions

1. Which one of the following is **correctly** matched with the definition and approaches of AI
A) Acting humanly - The rational agent approach

- B) Thinking humanly - The social constructive modelling approach
- C) Thinking rationally - The “laws of thought” approach
- D) Acting rationally - The Turing test approach

2. Artificial Intelligence is about

- A) Making a machine Intelligent
- B) Playing a game on Computer
- C) Programming on Machine with your Own Intelligence
- D) Putting your intelligence in Machine

3. One of the following is/are false about human vs. machine intelligence

- A) Machines perceive by set of rules rather than patterns
- B) Machines can label/figure out a missing part of a given object more effectively than humans
- C) Humans recall and store information by patterns rather than algorithms
- D) Machines can't think out of the box but humans do

4. One of the following is/are true about the benefits of Artificial intelligence except

- A) Useful for risky areas
- B) High speed and accuracy
- C) Increase dependency on machines
- D) Reliability
- E) All of them

5. Which of the following is not a goal of AI?

- A) Thinking humanly
- B) Adapting to the environment and situations
- C) Real Life Problem Solving
- D) To rule over humans

6. One of the following is/are true about dataset in machine learning systems except?

- A) Qualified/Clear dataset is enough even if its amount is small
- B) Invalid dataset gives invalid results
- C) Quality of dataset is better than quality of algorithms that you select
- D) Handling missing data is preferable because it has its own meanings

7. "Artificial Intelligence means to mimic a human. Hence, if a robot can move from one place to another like a human, then it comes under Artificial Intelligence."

- A) True B) False C) May be true or false

Chapter Two: Intelligent Agents

Objectives

After completing this chapter, students will be able to know and understand

- Introduction to Intelligent Agents
- Agents and the environments
- Rationality Vs Omniscience
- Structure of Intelligent Agents
- Autonomy
- Task Environments
- Properties of Task environments
- PEAS examples
- Agent Types

2.1 Introduction to Intelligent Agents

- Terminology
 - Performance Measure of Agent -It is the criteria, which determines how successful an agent is.
 - Behavior of Agent – It is the action that agent performs after any given sequence of percepts.
 - Percept – It is agent's perceptual inputs at a given instance.
 - Percept Sequence – It is the history of all that an agent has perceived till date.
 - Agent Function – It is a map from the precept sequence to an action.
- **An agent** is anything that can perceive its environment through **sensors** and acts upon that environment through effectors. An agent perceives its environment through sensors.
- The complete set of inputs at a given time is called a percept.
- The current percept or a sequence of precepts can influence the actions of an agent.
- The agent can change the environment through actuators or effectors.
- An operation involving an effector is called an action.
- Actions can be grouped into action sequences.
- The agent can have goals which it tries to achieve. Thus, an agent can be looked upon as a system that implements a mapping from percept sequences to actions.
- A performance measure has to be used in order to evaluate an agent.
- An autonomous agent decides autonomously which action to take in the current situation to maximize progress towards its goals.

- An Intelligent Agent must sense, must act, and must be autonomous (to some extent). It also must be rational.
- AI is about building rational agents.
- An agent is something that perceives and acts.
- A rational agent always does the right thing.
 - What are the functionalities (goals)?
 - What are the components?
 - How do we build them?
- **Agent performance**
 - An agent function implements a mapping from perception history to action.
 - The behaviour and performance of intelligent agents have to be evaluated in terms of the agent function.
 - The ideal mapping specifies which actions an agent ought to take at any point in time.
- The performance measure is a subjective measure to characterize how successful an agent is.
- The success can be measured in various ways.
 - It can be measured in terms of speed or efficiency of the agent.
 - It can be measured by the accuracy or the quality of the solutions achieved by the agent.
 - It can also be measured by power usage, money, etc.

2.2 Agents and the environments

- An AI system is composed of an **agent** and its environment.
- The **agents** act in their environment. The environment may contain other agents.
- An agent is anything that can perceive its environment through sensors and acts upon that environment through effectors.

2.3 Rationality Vs Omniscience

Rationality

- Rationality is nothing but status of being reasonable, sensible, and having good sense of judgment.
- Rationality is concerned with expected actions and results depending upon what the agent has perceived.
- Performing actions with the aim of obtaining useful information is an important part of rationality.
- Example of agent:
- Chess agent
 - World knowledge is the board state (all the pieces)
 - Sensory information is the opponents move
 - It's moves also change the board state

- Human: sensor (eye, ears, skins, taste). effectors (Hand, fingers, legs, mouth)
- Robot: sensor (camera, infrared,..), effectors (gripper, wheel, speaker,..)
- Software agent: function as sensor and actuator

Ideal Rational Agent

- An ideal rational agent is the one, which is capable of doing expected actions to maximize its performance measure, on the basis of –
 - Its percept sequences
 - Its built-in knowledge bases
- Rationality of an agent depends on the following –
 - The performance measures, which determine the degree of success.
 - Agent's Percept Sequence till now.
 - The agent's posterior knowledge about the environment.
 - The actions that the agent can carry out.
- A rational agent always performs right action, where the right action means the action that causes the agent to be most successful in the given percept sequence.
- Omniscience
 - It is the state of possessing unlimited knowledge about all things possible.
 - It is the capacity of knowing unlimited knowledge of all things that can be known.
 - It knows the actual effects of its actions and it is impossible in real world.
- ➔ A rational agent behaves according to its precepts and knowledge and attempts to maximize the expected performance.

2.4 Structure of Intelligent Agents

- The Structure of Intelligent Agents - Agent's structure can be viewed as –
 - Agent = Architecture + Agent Program
 - Architecture = the machinery that an agent executes on.
 - Agent Program = an implementation of an agent function.

2.5 Autonomy

- The **autonomy** of an agent is the extent to which its behaviour is determined by its **own experience**, rather than knowledge of designer.
- Extremes
 - No autonomy – ignores environment/data
 - Complete autonomy – must act randomly/no program
- Ideal: design agents to have some autonomy
 - Possibly become more autonomous with experience

2.6 Task Environments

- An environment in artificial intelligence is the surrounding of the agent.
- The environment is where agent lives, operate and provide the agent with something to sense and act upon it.
- The agent takes input from the environment through **sensors** and delivers the output to the environment through **actuators**.
- The most famous **artificial environment** is the **Turing Test environment**, in which one real and other artificial agent are tested on equal ground.

2.7 Properties of Task environments

- ✓ **Discrete / Continuous** – If there are a limited number of distinct, clearly defined, states of the environment, the environment is discrete (For example, chess); otherwise it is continuous (For example, driving).
- ✓ **Observable / Partially Observable** – If it is possible to determine the complete state of the environment at each time point from the percepts it is observable; otherwise it is only partially observable.
- ✓ **Static / Dynamic** – If the environment does not change while an agent is acting, then it is static; otherwise it is dynamic.
- ✓ **Single agent / Multiple agents** – The environment may contain other agents which may be of the same or different kind as that of the agent.
- ✓ **Accessible / Inaccessible** – If the agent's sensory apparatus can have access to the complete state of the environment, then the environment is accessible to that agent.
- ✓ **Deterministic / Non-deterministic** – If the next state of the environment is completely determined by the current state and the actions of the agent, then the environment is deterministic; otherwise it is non-deterministic.
- ✓ **Episodic / Non-episodic** – In an episodic environment, each episode consists of the agent perceiving and then acting. The quality of its action depends just on the episode itself. Subsequent episodes do not depend on the actions in the previous episodes. Episodic environments are much simpler because the agent does not need to think ahead.

2.8 PEAS examples

- There are four elements to take account when designing an agent to solve a particular problem
- PEAS
 - **Performance:** measure the success of an agent's behavior
 - **Environment:** where an agent operates

- **Actuator:** agent acts with in its environment
- **Sensor:** the agent senses its environment
- **Example 1: Agent - Taxi driver**
 - **P:** safe, fast, legal, maximize profit
 - **E:** road, other traffic, customer, pedestrian
 - **A:** brake, accelerator, signal, horn, steering wheel
 - **S:** camera, sonar, speedometer, GPS, odometer, engine sensor, keyboard.

2.9 Agent Types

Agents can be grouped into five classes based on their degree of perceived intelligence and capability. All these agents can improve their performance and generate better action over the time.

2.9.1 Simple reflex agent

- They are the simplest agents that take decisions on the basis of the **current percept** and ignore the rest of the percept history. These agents only succeed in the fully observable environment.
- The Simple reflex agent **does not consider any part of percept history during their decision and action process.**
- The Simple reflex agent works on **Condition-action rule**, which means it maps the current state to action.
- Such as a Room Cleaner agent, it works only if there is dirt in the room.
- Problems for the simple reflex agent design approach:
 - They have very limited intelligence
 - They do not have knowledge of non-perceptual parts of the current state
 - Not adaptive to changes in the environment.

2.9.2 Model-based reflex agent

- The Model-based agent can work in a partially observable environment, and track the situation.
- A model-based agent has two important factors:
 - **Model:** It is knowledge about "how things happen in the world," so it is called a Model-based agent.
 - **Internal State:** It is a representation of the current state based on percept history.
- These agents have the model, "which is knowledge of the world" and based on the model they perform actions.
- Updating the agent state requires information about:
 - How the world evolves
 - How the agent's action affects the world.

2.9.3 Goal-based agent

- The knowledge of the current state environment is not always sufficient to decide for an agent to what to do.
- The agent needs to know its **goal** which describes desirable situations.
- Goal-based agents expand the capabilities of the model-based agent by having the "**goal**" **information**.
- They choose an action, so that they can achieve the goal.
- These agents may have to consider **a long sequence of possible actions before deciding whether the goal is achieved or not**. Such considerations of different scenario are called **searching** and **planning**, which makes an agent proactive.

2.9.4 Utility-based agent

- These agents are similar to the goal-based agent but provide **an extra component of utility measurement** which makes them different by providing a measure of success at a given state.
- Utility-based agent act based not only goals but also the best way to achieve the goal.
- The Utility-based agent is useful when there are multiple possible alternatives, and an agent has to choose in order to perform the best action.
- The utility function maps each state to a real number to check how efficiently each action achieves the goals.

2.9.5 Learning agent

- Learning allows an agent to operate in initially unknown environments.
- The learning element modifies the performance element.
- Learning is required for true autonomy
- A learning agent in AI is the type of agent which can learn from its past experiences, or it has learning capabilities.
- It starts to act with basic knowledge and then able to act and adapt automatically through learning.
- A learning agent has mainly four conceptual components, which are:
 - **Learning element** is for making improvements by learning from environment
 - **Critic** Learning element takes feedback from critic which describes that how well the agent is doing with respect to a fixed performance standard.
 - **Performance element** is responsible for selecting external action
 - **Problem generator** is responsible for suggesting actions that will lead to new and informative experiences.

- Hence, learning agents are able to learn, analyse performance, and look for new ways to improve the performance.

2.10 Summery

- ✓ AI is to build intelligent agents that act so as to optimize performance.
- ✓ An agent perceives and acts in an environment, has architecture, and is implemented by an agent program.
- ✓ An ideal agent always chooses the action which maximizes its expected performance, given its percept sequence so far.
- ✓ An autonomous agent uses its own experience rather than built-in knowledge of the environment by the designer.
- ✓ An agent program maps from percept to action and updates its internal state.
 - ✓ Reflex agents respond immediately to precepts.
 - ✓ Goal-based agents act in order to achieve their goal(s).
 - ✓ Utility-based agents maximize their own utility function.
- ✓ Representing knowledge is important for successful agent design.
- ✓ The most challenging environments are partially observable, stochastic, sequential, dynamic, and continuous, and contain multiple intelligent agents.

2.11 Questions

- Which of the following is the meaning of the agent's percept sequence?**
 - A complete history of perceived things
 - A complete history of the actuator
 - Used to perceive the environment
 - None of these
- _____ are very helpful in AI for perceiving and acting upon the environment?**
 - Perceiver
 - Sensors
 - Sensors and Actuators
 - None of these
- Which of the following is the composition for agents in AI?**
 - Program
 - Architecture
 - Both A and B
 - None of these
- Artificial Intelligence is unable to do yet?**
 - Understanding the natural language robustly
 - Web mining
 - plans Construction in real-time dynamic systems
 - All of these

- E. None of these
5. **Problem generator is present in which of the following agent?**
- A. Reflex
B. Observing
C. Learning
D. None of these
6. **Which of the following is the rule of a simple reflex agent?**
- A. Simple-action rule
B. Simple & Condition-action rule
C. Condition-action rule
D. None of these
7. **We can improve the performance of agents with the help of?**
- A. Learning
B. Perceiving
C. Observing
D. None of these
8. **We can achieve the agent's goal with the help of action sequences?**
- A. Search
B. Plan
C. Retrieve
D. Both A and B
E. None of these
9. **The action of the Simple reflex agent fully depends upon which of the following?**
- A. Perception history
B. Current perception
C. Learning theory
D. Utility functions
E. None of these

Chapter Three: Problem Solving (Goal Based) Agents

Objectives

- Problem Solving by Searching
- What is search and terminologies?
- Search Strategies
- Unformed Search Strategies
 - DFS, BFS, Uniform cost search, DLS, Iterative Deeping DFS, and Bi-directional search.
- Informed Search Strategies
 - A star search, Best first search
- Local Search Strategies
 - Hill climbing search, Means ends analysis, Simulated annealing

- Adversarial Search Strategies
 - Mini-max algorithm, Alpha beta pruning
- Avoiding Repeated States
- Constraint Satisfaction Search

3.1 Problem Solving by Searching

3.2 What is search and terminologies?

Searching is a step by step procedure to solve a search-problem in a given search space. A search problem can have three main factors:

- **Search Space:** Search space represents a set of possible solutions, which a system may have.
- **Start State:** It is a state from where agent begins the search.
- **Goal test:** It is a function which observe the current state and returns whether the goal state is achieved or not.

Search tree: A tree representation of search problem is called Search tree. The root of the search tree is the root node which is corresponding to the initial state.

Actions: It gives the description of all the available actions to the agent.

Transition model: A description of what each action do, can be represented as a transition model.

Path Cost: It is a function which assigns a numeric cost to each path.

Solution: It is an action sequence which leads from the start node to the goal node.

Optimal Solution: If a solution has the lowest cost among all solutions.

3.5 Search Strategies

3.5.1 Unformed Search Strategies

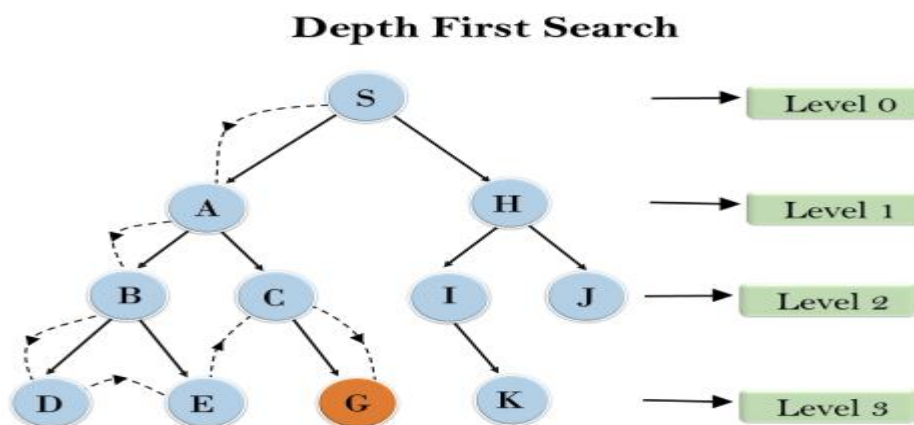
- The uninformed search does not contain any domain knowledge such as closeness, the location of the goal. It is also called level order traversal.
- It operates in a brute-force way as it only includes information about how to traverse the tree and how to identify leaf and goal nodes.
- Uninformed search applies a way in which search tree is searched without any information about the search space like initial state operators and test for the goal, so it is also called blind search.
- It examines each node of the tree until it achieves the goal node.

1. DFS

- Depth-first search is a recursive algorithm for traversing a tree or graph data structure.
- It is called the depth-first search because it starts from the root node and follows each path to its greatest depth node before moving to the next path.
- DFS uses a stack data structure for its implementation.
- The process of the DFS algorithm is similar to the BFS algorithm.

- Note: Backtracking is an algorithm technique for finding all possible solutions using recursion. It traverses a tree from Root node--->Left node ----> right node.
- **Advantage:**
 - DFS requires very less memory as it only needs to store a stack of the nodes on the path from root node to the current node.
 - It takes less time to reach to the goal node than BFS algorithm (if it traverses in the right path).
- **Disadvantage:**
 - There is the possibility that many states keep re-occurring, and there is no guarantee of finding the solution.
 - DFS algorithm goes for deep down searching and sometime it may go to the infinite loop.

Example



- **Completeness:** DFS search algorithm is complete within finite state space as it will expand every node within a limited search tree.
- **Time Complexity:** Time complexity of DFS will be equivalent to the node traversed by the algorithm. $T(n) = O(n^m)$, where m = maximum depth of any node and this can be much larger than d (Shallowest solution depth)
- **Space Complexity:** DFS algorithm needs to store only single path from the root node, hence space complexity of DFS is equivalent to the size of the fringe set, which is $O(bm)$.
- **Optimal:** DFS search algorithm is non-optimal, as it may generate a large number of steps or high cost to reach to the goal node.

2. BFS

- BFS algorithm starts searching from the root node of the tree and expands all successor node at the current level before moving to nodes of next level.

- It searches breadthwise in a tree or graph, so it is called breadth-first search.
- Breadth-first search implemented using FIFO queue data structure.
- If branching factor (average number of child nodes for a given node) = b and depth = d , then number of nodes at level $d = b^d$.
- The total no of nodes created in worst case is $b + b^2 + b^3 + \dots + b^d$.

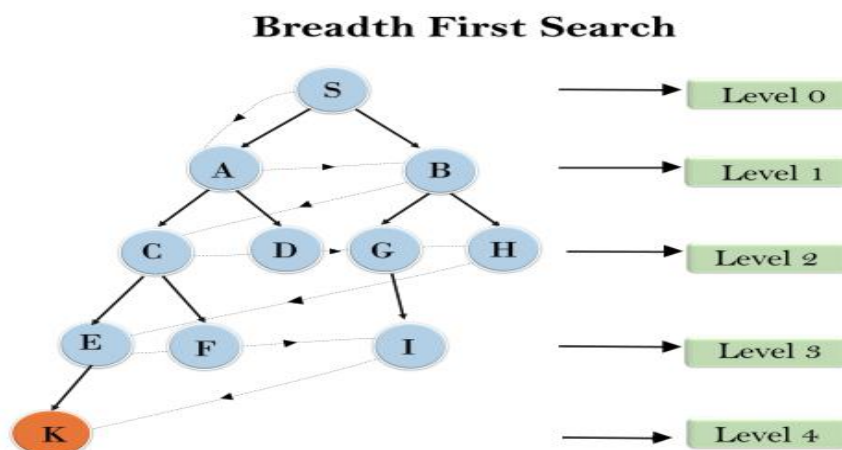
Advantages

- BFS will provide a solution if any solution exists.
- If there are more than one solution for a given problem, then BFS will provide the minimal solution which requires the least number of steps.

Disadvantages:

- It requires lots of memory since each level of the tree must be saved into memory to expand the next level.
- BFS needs lots of time if the solution is far away from the root node.

Example



Time Complexity: is obtained by the number of nodes traversed in BFS until the shallowest Node. $T(b) = O(b^d)$ where d = depth of shallowest solution and b is a node at every state.

Space Complexity: Space complexity of BFS algorithm is given by the Memory size of frontier which is $O(b^d)$

Completeness: BFS is complete, which means if the shallowest goal node is at some finite depth, then BFS will find a solution.

Optimality: BFS is optimal if path cost is a non-decreasing function of the depth of the node.

3. Depth limited search

A depth-limited search algorithm is similar to depth-first search with a predetermined limit. Depth-limited search can solve the drawback of the infinite path in the Depth-first search. In this algorithm, the node at the depth limit will treat as it has no successor nodes further.

Depth-limited search can be terminated with two Conditions of failure:

- Standard failure value: It indicates that problem does not have any solution.
- Cutoff failure value: It defines no solution for the problem within a given depth limit.

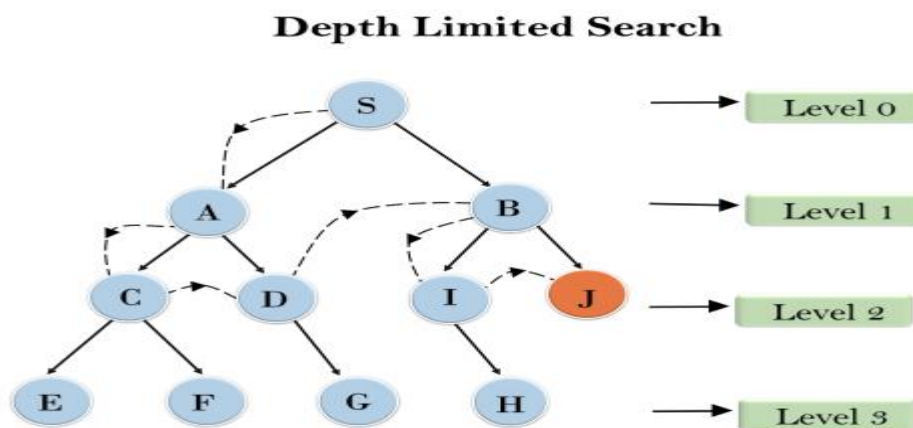
- **Advantages:**

- Depth-limited search is Memory efficient.

- **Disadvantages:**

- Depth-limited search also has a disadvantage of incompleteness.
- It may not be optimal if the problem has more than one solution.

- Example: Find the path to get the goal 'J' if depth limit is 2

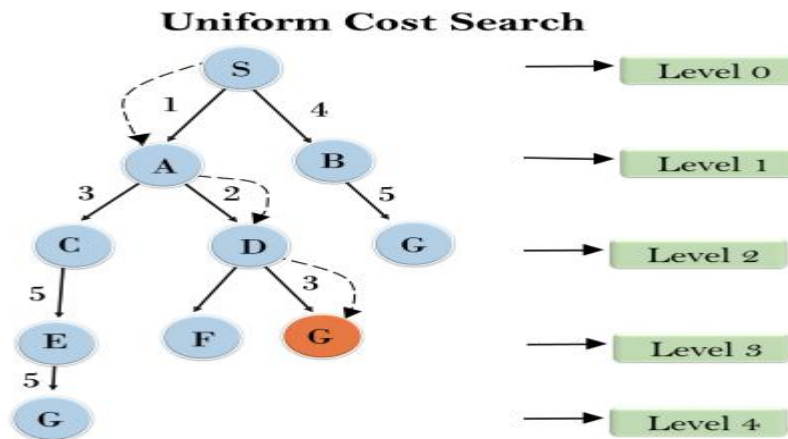


- **Completeness:** DLS search algorithm is complete if the solution is above the depth-limit.
- **Time Complexity:** Time complexity of DLS algorithm is $O(b^l)$.
- **Space Complexity:** Space complexity of DLS algorithm is $O(b \times l)$.
- **Optimal:** Depth-limited search can be viewed as a special case of DFS, and it is also not optimal even if $l > d$.

4. Uniform cost search

- Uniform-cost search is a searching algorithm used for traversing a weighted tree or graph.
- This algorithm comes into play when a different cost is available for each edge.
- The primary goal of the uniform-cost search is to find a path to the goal node which has the lowest cumulative cost.
- Uniform-cost search expands nodes according to their path costs from the root node.
- It can be used to solve any graph/tree where the optimal cost is in demand. A uniform-cost search algorithm is implemented by the priority queue.
- It gives maximum priority to the lowest cumulative cost.
- Uniform cost search is equivalent to BFS algorithm if the path cost of all edges is the same.

- **Advantages:**
 - Uniform cost search is optimal because at every state the path with the least cost is chosen.
- **Disadvantages:**
 - It does not care about the number of steps involve in searching and only concerned about path cost. Due to which this algorithm may be stuck in an infinite loop.
- Example : Find the goal node 'G' for the following search tree



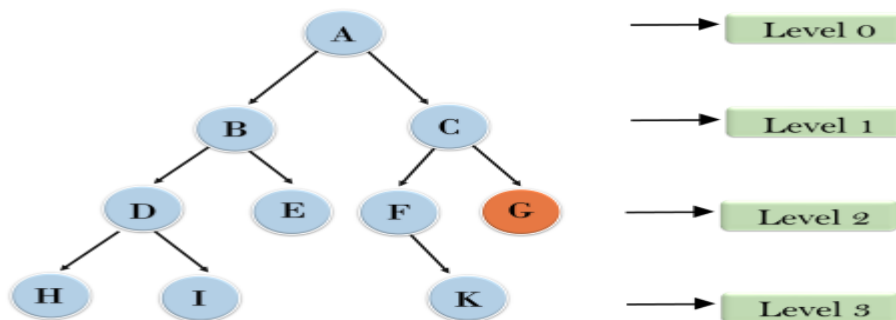
- **Completeness** - Uniform-cost search is complete, such as if there is a solution, UCS will find it.
- **Time Complexity** - Let C^* is Cost of the optimal solution, and ϵ is each step to get closer to the goal node. Then the number of steps is $= C^*/\epsilon + 1$. Here we have taken +1, as we start from state 0 and end to C^*/ϵ . Hence, the worst-case time complexity of Uniform-cost search is $O(b^{1 + \lceil C^*/\epsilon \rceil})$.
- **Space Complexity** - The same logic is for space complexity so, the worst-case space **complexity** of Uniform-cost search is $O(b^{1 + \lceil C^*/\epsilon \rceil})$.
- **Optimal** - Uniform-cost search is always optimal as it only selects a path with the lowest path cost.

5. Iterative Deeping DFS

- It is a combination of DFS and BFS algorithms. This search algorithm finds out the best depth limit and does it by gradually increasing the limit until a goal is found.
- This algorithm performs depth-first search up to a certain "depth limit", and it keeps increasing the depth limit after each iteration until the goal node is found.
- This Search algorithm combines the benefits of Breadth-first search's fast search and depth-first search's memory efficiency.
- The iterative search algorithm is useful uninformed search when search space is large, and depth of goal node is unknown.
- **Advantages:**
 - It combines the benefits of BFS and DFS search algorithm in terms of fast search and memory efficiency.
- **Disadvantages:**

- The main drawback of IDDFS is that it repeats all the work of the previous phase.
- Example : find the goal node G

Iterative deepening depth first search



⇒ Solution

1'st Iteration-----> A

2'nd Iteration----> A, B, C

3'rd Iteration----->A, B, D, E, C, F, G

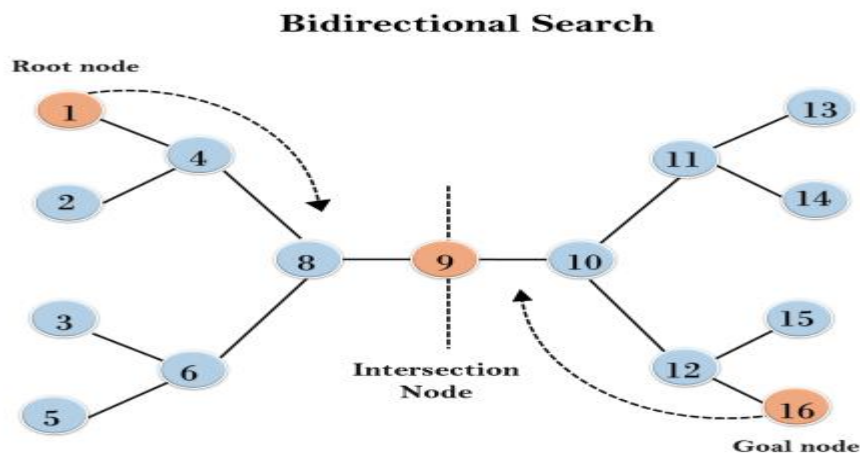
In the third iteration, the algorithm will find the goal node.

- Completeness - This algorithm is complete if the branching factor is finite.
- Time Complexity - Let's suppose b is the branching factor and depth is d then the worst-case time complexity is $O(b^d)$.
- Space Complexity - The space complexity of IDDFS will be $O(bd)$.
- Optimal - IDDFS algorithm is optimal if path cost is a non- decreasing function of the depth of the node.

6. Bidirectional search

- Bidirectional search algorithm runs two simultaneous searches, one from initial state called as forward-search and other from goal node called as backward-search, to find the goal node.
- Bidirectional search replaces one single search graph with two small sub-graphs in which one starts the search from an initial vertex and other starts from goal vertex.
- The search stops when these two graphs intersect each other.
- Bidirectional search can use search techniques such as BFS, DFS, DLS, etc.
- **Advantages:**
 - Bidirectional search is fast.
 - Bidirectional search requires less memory
- **Disadvantages:**

- Implementation of the bidirectional search tree is difficult.
- In bidirectional search, one should know the goal state in advance.
- Example - In the below search tree, bidirectional search algorithm is applied. This algorithm divides one graph/tree into two sub-graphs. It starts traversing from node 1 in the forward direction and starts from goal node 16 in the backward direction.
- The algorithm terminates at node 9 where two searches meet.
- Find the path to get the goal node '16'



- **Completeness:** Bidirectional Search is complete if we use BFS in both searches.
- **Time Complexity:** Time complexity of bidirectional search using BFS is $O(b^d)$.
- **Space Complexity:** Space complexity of bidirectional search is $O(b^d)$.
- **Optimal:** Bidirectional search is Optimal.

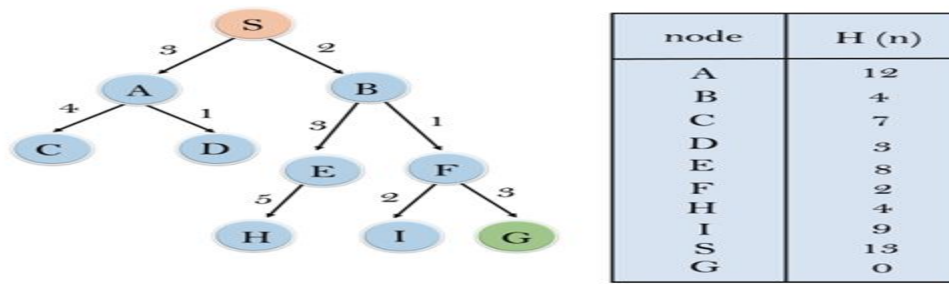
3.5.2 Informed Search Strategies

- It uses domain knowledge. In an informed search, problem information is available which can guide the search.
- Informed search strategies can find a solution more efficiently than an uninformed search strategy.
- Informed search is also called a Heuristic search. A heuristic is a way which might not always be guaranteed for best solutions but guaranteed to find a good solution in reasonable time.
- Informed search can solve much complex problem which could not be solved in another way.
- Heuristics function: Heuristic is a function which is used in Informed Search, and it finds the most promising path.
- It takes the current state of the agent as its input and produces the estimation of how close agent is from the goal.
- The heuristic method, however, might not always give the best solution, but it guaranteed to find a good solution in reasonable time.

- Heuristic function estimates how close a state is to the goal. It is represented by $h(n)$, and it calculates the cost of an optimal path between the pair of states. Its value is always positive.
- Admissibility of the heuristic function is given as: $h(n) \leq h^*(n)$ Here $h(n)$ is heuristic cost, and $h^*(n)$ is the estimated cost. Hence heuristic cost should be less than or equal to the estimated cost.
- Pure Heuristic Search - Pure heuristic search is the simplest form of heuristic search algorithms. It expands nodes based on their heuristic value $h(n)$.

1. Best first search

- Greedy best-first search algorithm always selects the path which appears best at that moment.
- It is the combination of depth-first search and breadth-first search algorithms. It uses the heuristic function and search.
- Best-first search allows us to take the advantages of both algorithms. With the help of best-first search, at each step, we can choose the most promising node.
- In the best first search algorithm, we expand the node which is closest to the goal node and the closest cost is estimated by heuristic function-
- **$f(n) = g(n) + h(n)$. Where, $h(n)$ = estimated cost from node n to the goal.**
- **It uses the priority queue.**
- **Best first search algorithm steps**
 - Step 1: Place the starting node into the OPEN list.
 - Step 2: If the OPEN list is empty, Stop and return failure.
 - Step 3: Remove the node n , from the OPEN list which has the lowest value of $h(n)$, and places it in the CLOSED list.
 - Step 4: Expand the node n , and generate the successors of node n .
 - Step 5: Check each successor of node n , and find whether any node is a goal node or not. If any successor node is goal node, then return success and terminate the search, else proceed to Step 6.
 - Step 6: For each successor node, algorithm checks for evaluation function $f(n)$, and then check if the node has been in either OPEN or CLOSED list. If the node has not been in both list, then add it to the OPEN list.
 - Step 7: Return to Step 2.
- Example - Consider the below search problem, and traverse it using greedy best-first search to get the goal node 'G'



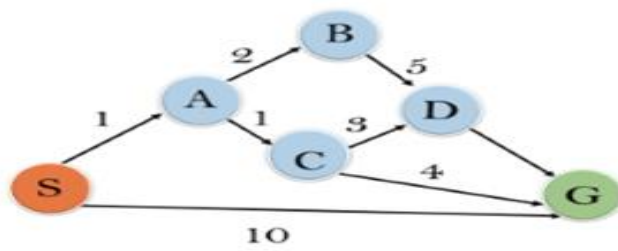
- **Expand the nodes of S and put in the CLOSED list**
 - Initialization: Open [A, B], Closed [S]
 - Iteration 1: Open [A], Closed [S, B]
 - Iteration 2: Open [E, F, A], Closed [S, B]
 - : Open [E, A], Closed [S, B, F]
 - Iteration 3: Open [I, G, E, A], Closed [S, B, F]
 - : Open [I, E, A], Closed [S, B, F, G]
 - Hence the final solution path will be: S----> B----->F----> G
- **Advantages:**
 - Best first search can switch between BFS and DFS by gaining the advantages of both the algorithms.
 - This algorithm is more efficient than BFS and DFS algorithms.
- **Disadvantages:**
 - It can behave as an unguided depth-first search in the worst-case scenario.
 - It can get stuck in a loop as DFS.
 - This algorithm is not optimal.
- **Time Complexity:** The worst case time complexity of Greedy best first search is $O(b^m)$.
- **Space Complexity:** The worst case space complexity of Greedy best first search is $O(b^m)$, Where, m is the maximum depth of the search space.
- **Complete:** Greedy best-first search is also incomplete, even if the given state space is finite.
- **Optimal:** Greedy best first search algorithm is not optimal.

2. A * search / A star search

- A* search is the most commonly known form of best-first search.
- It uses heuristic function $h(n)$, and cost to reach the node n from the start state $g(n)$. It has combined features of UCS and greedy best-first search, by which it solves the problem efficiently.

- A* search algorithm finds the shortest path through the search space using the heuristic function. This search algorithm expands less search tree and provides optimal result faster. A* algorithm is similar to UCS except that it uses $g(n)+h(n)$ instead of $g(n)$.
- In A* search algorithm, we use search heuristic as well as the cost to reach the node. Hence, we can combine both costs as following, and this sum is called as a fitness number.
- At each point in the search space, only those nodes are expanded which have the lowest value of $f(n)$, and the algorithm terminates when the goal node is found.
- Advantages:
 - A* search algorithm is the best algorithm than other search algorithms.
 - A* search algorithm is optimal and complete.
 - This algorithm can solve very complex problems.
- Disadvantages:
 - It does not always produce the shortest path as it mostly based on heuristics and approximation.
 - A* search algorithm has some complexity issues.
 - The main drawback of A* is memory requirement as it keeps all generated nodes in the memory, so it is not practical for various large-scale problems.
- **Algorithm of A* search steps**
 - Place the starting node in the OPEN list.
 - Check if the OPEN list is empty or not, if the list is empty then return failure and stops.
 - Select the node from the OPEN list which has the smallest value of evaluation function ($g+h$), if node n is goal node then return success and stop, otherwise
 - Expand node n and generate all of its successors, and put n into the closed list. For each successor n' , check whether n' is already in the OPEN or CLOSED list, if not then compute evaluation function for n' and place into Open list.
 - Else if node n' is already in OPEN and CLOSED, then it should be attached to the back pointer which reflects the lowest $g(n')$ value.
 - Return to Step 2.

- Example - In this example, we will traverse the given graph using the A* algorithm.



State	$h(n)$
S	5
A	3
B	4
C	2
D	6
G	0

- The heuristic value of all states is given in the table so we will calculate the $f(n)$ of each state using the formula $f(n) = g(n) + h(n)$, where $g(n)$ is the cost to reach any node from start state.
- Here we will use OPEN and CLOSED list.
 - Initialization: $\{(S, 5)\}$
 - Iteration1: $\{(S \rightarrow A, 4), (S \rightarrow G, 10)\}$
 - Iteration2: $\{(S \rightarrow A \rightarrow C, 4), (S \rightarrow A \rightarrow B, 7), (S \rightarrow G, 10)\}$
 - Iteration3: $\{(S \rightarrow A \rightarrow C \rightarrow G, 6), (S \rightarrow A \rightarrow C \rightarrow D, 11), (S \rightarrow A \rightarrow B, 7), (S \rightarrow G, 10)\}$
 - Iteration 4 gives the final result, as $S \rightarrow A \rightarrow C \rightarrow G$ it provides the optimal path with cost 6.
- Complete: A* algorithm is complete as long as:
 - Branching factor is finite.
 - Cost at every action is fixed.
- Optimal: A* search algorithm is optimal if it follows below two conditions:
 - Admissible: the first condition requires for optimality is that $h(n)$ should be an admissible heuristic for A* tree search. An admissible heuristic is optimistic in nature.
 - Consistency: Second required condition is consistency for only A* graph-search.
- If the heuristic function is admissible, then A* tree search will always find the least cost path.
- Time Complexity: The time complexity of A* search algorithm depends on heuristic function, and the number of nodes expanded is exponential to the depth of solution d . So the time complexity is $O(b^d)$, where b is the branching factor.
- Space Complexity: The space complexity of A* search algorithm is $O(b^d)$.

3.5.3 Local Search Strategies

In many optimization problems, the path to the goal is irrelevant; the goal state itself is the solution.

- Local search: widely used for *very big* problems
- Returns good but *not optimal* solutions

State space = set of "complete" configurations

- Find configuration satisfying constraints
- Examples: n-Queens, VLSI layout, airline flight schedules

Local search algorithms

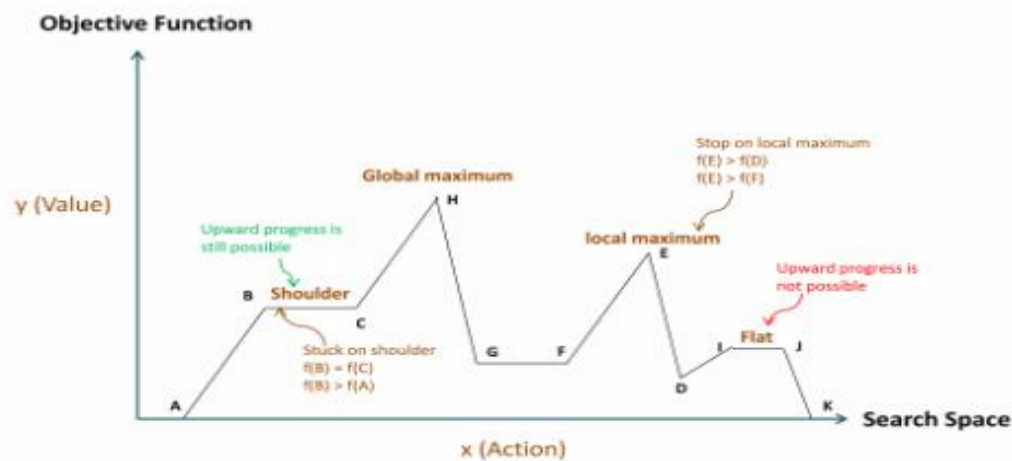
- Keep a single "current" state, or small set of states
- Iteratively try to improve it / them
- Very memory efficient
 - o keeps only one or a few states
 - o You control how much memory you use

1. Hill climbing algorithm

Hill climbing algorithm is a local search algorithm which continuously moves in the direction of increasing elevation/value to find the peak of the mountain or best solution to the problem. It terminates when it reaches a peak value where no neighbor has a higher value.

It is also called **greedy local search** as it only looks to its good immediate neighbor state and not beyond that. A node of hill climbing algorithm has two components which are **state** and **value**. Hill Climbing is mostly used when a good heuristic is available. In this algorithm, we don't need to maintain and handle the search tree or graph as it only keeps a single current state.

- Features of hill climbing algorithm
 - **Generate and Test variant:** The Generate and Test method produce feedback which helps to decide which direction to move in the search space.
 - **Greedy approach:** Hill-climbing algorithm search moves in the direction which optimizes the cost.
 - **No backtracking:** It does not backtrack the search space, as it does not remember the previous states.
- **The state-space landscape** is a graphical representation of the hill-climbing algorithm which is showing a graph between various states of algorithm and Objective function/Cost.
- **Local Maximum** - is a state which is better than its neighbor states, but there is also another state which is higher than it.
- **Global Maximum** - is the best possible state of state space landscape. It has the highest value of objective function.
- **Current state** - It is a state in a landscape diagram where an agent is currently present.
- **Flat local maximum** - It is a flat space in the landscape where all the neighbor states of current states have the same value.
- **Shoulder** - It is a plateau region which has an uphill edge.



⇒ Types of hill climbing algorithms

1. Simple hill climbing algorithm

- Simple hill climbing is the simplest way in which it only evaluates the neighbor node state at a time and selects the first one which optimizes current cost and set it as a current state.
- It only checks its one successor state, and if it finds better than the current state, then move else be in the same state.
- features
 - Less time consuming
 - Less optimal solution and the solution is not guaranteed
- Algorithmic steps
 - Create a CURRENT node, NEIGHBOUR node, and a GOAL node.
 - If the CURRENT node=GOAL node, return GOAL and terminate the search.
 - Else CURRENT node <= NEIGHBOUR node, move ahead.
 - Loop until the goal is not reached or a point is not found.

2. Steepest-Ascent algorithm

- The steepest-Ascent algorithm is a variation of simple hill climbing algorithm.
- It examines all the neighboring nodes of the current state and selects one neighbor node which is closest to the goal state.
- It consumes more time as it searches for multiple neighbours
- Both simple and steepest-ascent hill climbing search, fails when there is no closer node.
- **Algorithmic steps**
 1. Create a CURRENT node and a GOAL node.
 2. If the CURRENT node=GOAL node, return GOAL and terminate the search.
 3. Loop until a better node is not found to reach the solution.

4. If there is any better successor node present, expand it.
5. When the GOAL is attained, return GOAL and terminate.

3. Stochastic hill climbing:

- Stochastic hill climbing does not examine for all its neighbor before moving.
- Rather, this search algorithm selects one neighbor node at random and decides whether to choose it as a current state or examine another state.
- It does not focus on all the nodes.
- It selects one node at random and decides whether it should be expanded or search for a better one.

Algorithmic steps

- Evaluate the initial state. If it is a goal state then stop and return success. Otherwise, make the initial state the current state.
 - Repeat these steps until a solution is found or the current state does not change.
 - Select a state that has not been yet applied to the current state.
 - Apply the successor function to the current state and generate all the neighbor states.
 - Among the generated neighbor states which are better than the current state choose a state randomly
 - If the chosen state is the goal state, then return success, else make it the current state and repeat step 2.
- Exit from the function.

⇒ Problems of hill climbing algorithm

1. Local Maximum: A local maximum is a peak state in the landscape which is better than each of its neighboring states, but there is another state also present which is higher than the local maximum.

- **Solution:** Backtracking technique can be a solution of the local maximum in state space landscape. Create a list of the promising path so that the algorithm can backtrack the search space and explore other paths as well.

2. Plateau: A plateau is the flat area of the search space in which all the neighbor states of the current state contains the same value, because of this algorithm does not find any best direction to move. A hill-climbing search might be lost in the plateau area.

- **Solution:** The solution for the plateau is to take big steps or very little steps while searching, to solve the problem. Randomly select a state which is far away from the current state so it is possible that the algorithm could find non-plateau region.

3. Ridges: A ridge is a special form of the local maximum. It has an area which is higher than its surrounding areas, but itself has a slope, and cannot be reached in a single move.

- **Solution:** With the use of bidirectional search, or by moving in different directions, we can improve this problem.

2. Means ends analysis

Means-Ends Analysis is mixed strategy, it makes possible that first to solve the major part of a problem and then go back and solve the small problems arise during combining the big parts of the problem. It is a mixture of Backward and forward search technique. The MEA analysis computes by evaluating the **difference between the current state and goal state**. The means-ends analysis process can be applied recursively for a problem.

- **Steps.**
 - First, evaluate the difference between Initial State and final State.
 - Select the various operators which can be applied for each difference.
 - Apply the operator at each difference, which reduces the difference between the current state and goal state.

Algorithms

- **Step 1:** Compare CURRENT to GOAL, if there are no differences between both then return Success and Exit.
- **Step 2:** Else, select the most significant difference and reduce it by doing the following steps until the success or failure occurs.
 - Select a new operator O which is applicable for the current difference, and if there is no such operator, then signal failure.
 - Attempt to apply operator O to CURRENT. Make a description of two states.
 - i) O-Start, a state in which O's preconditions are satisfied.
 - ii) O-Result, the state that would result if O were applied In O-start.
 - If
 - (First-Part $\leftarrow$ MEA(CURRENT,O-START))
 - And
 - (LAST-Part $\leftarrow$ MEA (O-Result, GOAL)), are successful, then signal Success and return the result of combining FIRST-PART, O, and LAST-PART.

3.5.4 Adversarial Search Strategies

Adversarial search is a search, where we examine the problem which arises when we try to plan ahead of the world and other agents are planning against us. The environment with more than one agent is termed as **multi-agent environment**, in which each agent is an opponent of other agent and playing

against each other. Each agent needs to consider the action of other agent and effect of that action on their performance.

- So, **Searches in which two or more players with conflicting goals are trying to explore the same search space for the solution, are called adversarial searches, often known as Games.**
- Games are modeled as a Search problem and heuristic evaluation function, and these are the two main factors which help to model and solve games in AI.

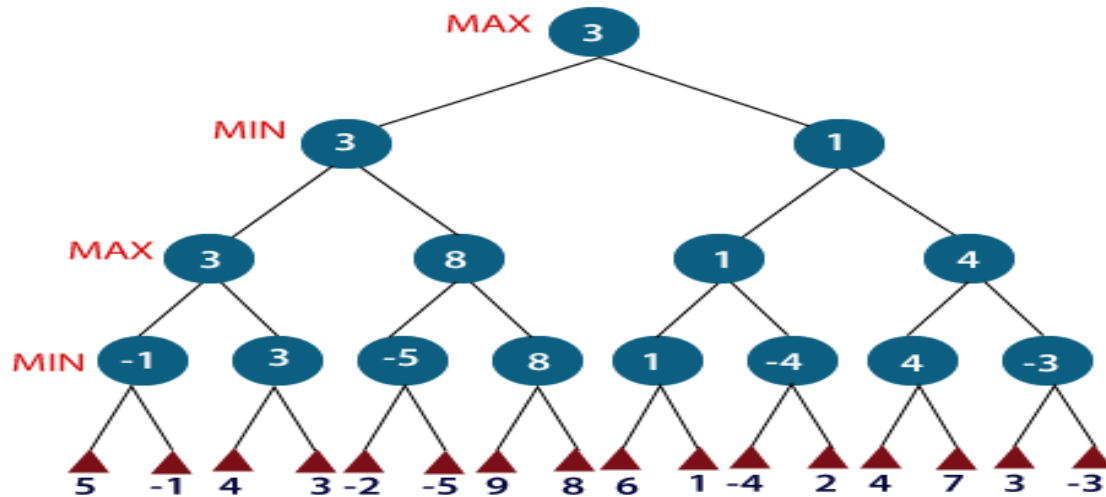
1. Minimax algorithm

Mini-max algorithm is a recursive or backtracking algorithm which is used in decision-making and game theory. It provides an optimal move for the player assuming that opponent is also playing optimally.

Mini-Max algorithm uses recursion to search through the game-tree. Min-Max algorithm is mostly used for game playing in AI such as Chess, Checkers, tic-tac-toe, go, and various tow-players game. In this algorithm two players play the game, one is called **MAX** and other is called **MIN**.

- Both the players fight it as the opponent player gets the minimum benefit while they get the maximum benefit.
- Both Players of the game are opponent of each other, where MAX will select the maximized value and MIN will select the minimized value.
- The minimax algorithm performs a **depth-first search algorithm** for the exploration of the complete game tree.
- The minimax algorithm proceeds all the way down to the terminal node of the tree, then backtrack the tree as the recursion.
- Players will be two
 - **MIN:** Decrease the chances of **MAX** to win the game.
 - **MAX:** Increases his chances of winning the game.
- MINIMAX algorithm is a backtracking algorithm where it backtracks to pick the best move out of several choices.
- MINIMAX strategy follows the **DFS (Depth-first search)** concept.
- Here, we have two players **MIN and MAX**, and the game is played alternatively between them, i.e., when **MAX** made a move, then the next turn is of **MIN**. It means the move made by MAX is fixed and, he cannot change it. The same concept is followed in DFS strategy, i.e., we follow the same path and cannot change in the middle. That's why in MINIMAX algorithm, instead of BFS, we follow **DFS**.
- Keep on generating the game tree/ search tree till a limit **d**.
- Compute the move using a heuristic function.

- Propagate the values from the leaf node till the current position following the minimax strategy.
- Make the best move from the choices.



- For example, in the above figure, the two players **MAX** and **MIN** are there. **MAX** starts the game by choosing one path and propagating all the nodes of that path.
- Now, **MAX** will backtrack to the initial node and choose the best path where his utility value will be the maximum.
- After this, it's **MIN** chance. **MIN** will also propagate through a path and again will backtrack, but **MIN** will choose the path which could minimize **MAX** winning chances or the utility value.
- So, if the level is minimizing, the node will accept the minimum value from the successor nodes.
- If the level is maximizing, the node will accept the maximum value from the successor.
- Minimax value of a node (backed up value):
- If N is terminal, use the utility value
- If N is a Max move, take max of successors
- If N is a Min move, take min of successors
- Choose the move with the highest minimax value
- best achievable payoff against best play
- Choose moves that will lead to a win, even though min is trying to block

2. Alpha Beta Pruning

Alpha-beta pruning is a modified version of the minimax algorithm. It is an optimization technique for the minimax algorithm. In the minimax search algorithm that the number of game states it has to examine are exponential in depth of the tree. Since we cannot eliminate the exponent, but we can cut it to half. Hence there is a technique by which without checking each node of the game tree we can compute the correct minimax decision, and this technique is called pruning or Alpha beta algorithm.

- Alpha-beta pruning can be applied at any depth of a tree, and sometimes it **not only prune the tree leaves but also entire sub-tree.**
- **Alpha:** The best (highest-value) choice we have found so far at any point along the path of Maximizer. The initial value of alpha is $-\infty$.
- **Beta:** The best (lowest-value) choice we have found so far at any point along the path of Minimizer. The initial value of beta is $+\infty$.
- It removes all the nodes which are not really affecting the final decision but making algorithm slow. Hence by pruning these nodes, it makes the algorithm fast.
- The main condition which required for alpha-beta pruning is $\alpha \geq \beta$
- The **Max player** will only **update the value of alpha.**
- The **Min player** will only **update the value of beta.**
- While backtracking the tree, the node values will be passed to upper nodes instead of values of alpha and beta.
- We will only pass the alpha, beta values to the child nodes.
- The effectiveness of alpha-beta pruning is highly dependent on the order in which each node is examined.
- **Worst ordering:** In some cases, alpha-beta pruning algorithm does not prune any of the leaves of the tree, and works exactly as minimax algorithm.
 - consumes more time
 - the best move occurs on the right side of the tree.
 - The time complexity for such an order is $O(b^m)$.
- **Ideal ordering:** occurs when lots of pruning happens in the tree, and best moves occur at the left side of the tree.
 - go deep twice as minimax algorithm in the same amount of time.
 - Complexity in ideal ordering is $O(b^{m/2})$.
- Rules to find good ordering:
 - Occur the best move from the shallowest node.
 - Order the nodes in the tree such that the best nodes are checked first.
 - Use domain knowledge while finding the best move.
 - We can bookkeep the states, as there is a possibility that states may repeat.

3.6 Avoiding Repeated States

- Do not return to the parent state (e.g., in 8 puzzle problem, do not allow the Up move right after a Down move)
- Do not create solution paths with cycles.

- Do not generate any repeated states (need to store and check a potentially large number of states)
- This is done by keeping a list of "expanded states" i.e., states whose daughters have already been put on the enqueued list. This entails removing states from the "enqueued list" and placing them on an "expanded list" (In the standard algorithm literature, the list of expanded states is called the "closed list ", thus, we would move states from the open list to the closed list)

3.7 Constraint Satisfaction Search

Constraint satisfaction is a technique where a problem is solved when its values satisfy certain constraints or rules of the problem. Such type of technique leads to a deeper understanding of the problem structure as well as its complexity.

Constraint satisfaction depends on three components, namely:

- **X:** It is a set of variables.
- **D:** It is a set of domains where the variables reside. There is a specific domain for each variable.
- **C:** It is a set of constraints which are followed by the set of variables.

In constraint satisfaction, domains are the spaces where the variables reside, following the problem specific constraints. These are the three main elements of a constraint satisfaction technique. The constraint value consists of a pair of **{scope, rel}**. The **scope** is a tuple of variables which participate in the constraint and **rel** is a relation which includes a list of values which the variables can take to satisfy the constraints of the problem.

Solving Constraint Satisfaction Problems

The requirements to solve a constraint satisfaction problem (CSP) is:

- A state-space
- The notion of the solution.

A state in state-space is defined by assigning values to some or all variables such as **{X₁=v₁, X₂=v₂, and so on...}**.

An assignment of values to a variable can be done in three ways:

- **Consistent or Legal Assignment:** An assignment which does not violate any constraint or rule is called Consistent or legal assignment.
- **Complete Assignment:** An assignment where every variable is assigned with a value, and the solution to the CSP remains consistent. Such assignment is known as Complete assignment.
- **Partial Assignment:** An assignment which assigns values to some of the variables only. Such type of assignments are called Partial assignments.

Types of Domains in CSP

There are following two types of domains which are used by the variables :

- **Discrete Domain:** It is an infinite domain which can have one state for multiple variables. **For example**, a start state can be allocated infinite times for each variable.
- **Finite Domain:** It is a finite domain which can have continuous states describing one domain for one specific variable. It is also called a continuous domain.

Constraint Types in CSP

With respect to the variables, basically there are following types of constraints:

- **Unary Constraints:** It is the simplest type of constraints that restricts the value of a single variable.
- **Binary Constraints:** It is the constraint type which relates two variables. A value x_2 will contain a value which lies between x_1 and x_3 .
- **Global Constraints:** It is the constraint type which involves an arbitrary number of variables.

Some special types of solution algorithms are used to solve the following types of constraints:

- **Linear Constraints:** These type of constraints are commonly used in linear programming where each variable containing an integer value exists in linear form only.
- **Non-linear Constraints:** These type of constraints are used in non-linear programming where each variable (an integer value) exists in a non-linear form.

Note: A special constraint which works in real-world is known as **Preference constraint**.

Constraint Propagation

In local state-spaces, the choice is only one, i.e., to search for a solution. But in CSP, we have two choices either:

- We can search for a solution or
- We can perform a special type of inference called **constraint propagation**.

Constraint propagation is a special type of inference which helps in reducing the legal number of values for the variables. The idea behind constraint propagation is **local consistency**.

In local consistency, variables are treated as **nodes**, and each binary constraint is treated as an **arc** in the given problem. **There are following local consistencies which are discussed below:**

- **Node Consistency:** A single variable is said to be node consistent if all the values in the variable's domain satisfy the unary constraints on the variables.
- **Arc Consistency:** A variable is arc consistent if every value in its domain satisfies the binary constraints of the variables.
- **Path Consistency:** When the evaluation of a set of two variable with respect to a third variable can be extended over another variable, satisfying all the binary constraints. It is similar to arc consistency.

- **k-consistency:** This type of consistency is used to define the notion of stronger forms of propagation. Here, we examine the k-consistency of the variables.

3.8 Sample Questions

1. **Which of the following is identical to the closed list in Graph search?**
 - (A). Transposition table
 - (B). Depth-first search
 - (C). Hill climbing search algorithm
 - (D). None of the above
2. **According to the minimax search algorithm, which of the following values are independent?**
 - (A). Root is independent
 - (B). Every state is dependent
 - (C). Pruned leaves x and y
 - (D). None of the above
3. **How we can increase the effectiveness of the alpha-beta pruning?**
 - (A). Depends on the order in which of the following they are executed
 - (B). Depends on the nodes
 - (C). All of these
 - (D). None of the above
4. **Which of the following function is used to find the feasibility of a complete game tree?**
 - (A). Transposition
 - (B). Evaluation function
 - (C). Alpha-beta pruning
 - (D). All of these
5. **Which of the following search removes the branches that can't influence the final decision, and it's equal to minimax search?**
 - (A). Depth-first
 - (B). Alpha-beta pruning
 - (C). Breadth-first
 - (D). None of the above
6. **Which of the following search is identical to minimax search?**
 - (A). Depth-first
 - (B). Hill-climbing
 - (C). Breadth-first
 - (D). All of these

7. **Where do the values of alpha-beta search can be modified?**
- (A). Initial state itself
 - (B). Along the path of search
 - (C). At the end
 - (D). None of the above
8. **According to the alpha-beta pruning, select the value assigned to alpha and beta?**
- (A). Alpha = max
 - (B). Beta = min
 - (C). Beta = max
 - (D). Both A and B
 - (E). None of the above
9. **One of the following is/are correctly matched except**
- A) Informed search – Best first search
 - B) A * search depends on heuristic value
 - C) Brute force strategies – Uniformed algorithms
 - D) Blind search – A star search
10. One of the evaluations of AI searching algorithm is 'its optimality', so what is optimality means
- A) It always finds a least cost solution for a problem
 - B) It always finds a solution of a problem if it exist
 - C) It always takes a small amount of time and space
 - D) It always finds a solution of a problem even if it does not exist

Chapter Four: Knowledge Representation and Reasoning

Objectives

- Knowledge based agent
- Architecture of Knowledge based agents
- Levels of knowledge
 - Logical level, knowledge level and implementation level
- Approaches to design KB agents
 - Procedural and declarative knowledge
- Knowledge Representation
- Techniques of Knowledge Representation

- Logical representation, Frame representation, Semantic network representation, and Production rule representation.
- Propositional Logic
- Logical connectives in PL
- Inference in Propositional Logic
- Predicate (First-Order) Logic
- Quantifiers in FOL (Universal and existential quantifiers)
- Inference in First-Order Logic
- Unification in FOL
- Resolution in FOL
- Reasoning
- Reasoning under uncertainty

4.1 Knowledge based agent

An intelligent agent needs knowledge about the real world for **taking decisions** and **reasoning** to act efficiently. Knowledge-based agents are agents who have the capability of maintaining an internal state of knowledge, reason over that knowledge, update their knowledge after observations and take actions. It is composed of two main parts: Knowledge-base, and Inference system.

Logical agent – agents with some representation of complex knowledge about the world / its environment and uses the inferences to derive new information from the knowledge combined with new inputs.

KB is a set of sentences in a formal language representing facts about the world.

Knowledge is a sentence in a knowledge representation language (formal language).

KB agent must able to do

- Should be able to represent states, actions...
- Should be able to incorporate new percepts.
- Can update the internal representation of the world.
- Can deduce the internal representation of the world,
- Can deduce appropriate actions.

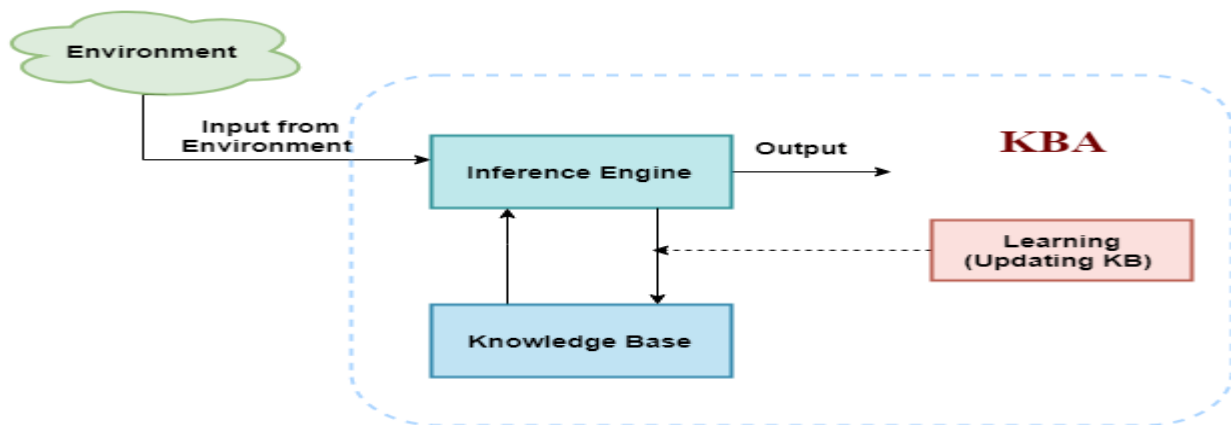
4.2 Architecture of Knowledge based agents

Knowledge-base (KB) is a central component of a knowledge-based agent. **KB** - is a collection of sentences ('sentence' is a technical term and it is not identical to sentence in English). The Knowledge-base of **KBA** stores fact about the world.

Knowledge-base is required for updating knowledge for an agent to learn with experiences and take action as per the knowledge.

Inference - Deriving new sentences from old. **Inference system** allows us to add a new sentence to the knowledge base. A sentence is a proposition about the world. Inference system applies logical rules to the KB to deduce new information. Inference system generates new facts so that an agent can update the KB. An inference system works mainly in two rules:

- **Forward chaining (facts to goals, data driven)**
- **Backward chaining (goals to facts, goal driven)**



4.3 Levels of knowledge

1. Knowledge level

- Knowledge level is the first level of knowledge-based agent.
- In this level we need to specify **what the agent knows**, and **what the agent goals** are.
- For example, suppose an automated taxi agent needs to go from a station A to station B, and he knows the way from A to B, so this comes at the knowledge level.

2. Logical level:

- How the knowledge representation of knowledge is stored.
- In this level, sentences are encoded into different logics.
- At this level, an encoding of knowledge into logical sentences occurs.
- Example - the automated taxi agent to reach to the destination B.

3. Implementation level:

- It is the physical representation of logic and knowledge.
- In this level agent **perform actions** as per logical and knowledge level.
- Example - an automated taxi agent actually implement his knowledge and logic so that he can reach to the destination.

4.4 Approaches to design KB agents

There are mainly two approaches to build a knowledge-based agent:

1. Declarative approach:

- Feeding the necessary information in an empty knowledge-based system.
- Such type of approach is used to design a knowledge-based system.
- The agent designer **TELLS** sentences to the empty system one by one until the system becomes knowledgeable enough to deal with the environment.

2. Procedural approach

- Knowledge is stored into an empty system in the form of program code.
- It designs the behavior of the system via coding.
- Directly encode desired behavior as a program code.
 - Just need to write a program that already encodes the desired behavior or agent.
- But a successful agent can be built by combining both **declarative and procedural approaches**, and declarative knowledge can often be compiled into more efficient procedural code.

4.5 Knowledge Representation

It is responsible for representing information about the real world so that a **computer** can understand and can utilize this knowledge to **solve the complex real world problems**

KR is a way which describes how we can represent knowledge in artificial intelligence. Knowledge representation is not just storing data into some database, but it also enables an intelligent machine to **learn from that knowledge and experiences** so that it can behave **intelligently like a human**.

Following are the kind of knowledge which needs to be represented in AI systems:

- **Object:** All the facts about objects in our world domain.
- **Events:** Events are the actions which occur in our world.
- **Performance:** It describe behavior which involves knowledge about how to do things.
- **Meta-knowledge:** It is knowledge about what we know.
- **Facts:** Facts are the truths about the real world and what we represent.

4.6 Techniques of Knowledge Representation

There are mainly four ways of knowledge representation which are given as follows:

1. Logical Representation

- LP is a language with some concrete rules which deals with propositions and has no ambiguity in representation.
- LP - drawing a conclusion based on various conditions.
- It consists of precisely defined syntax and semantics which supports the sound inference.

- Each sentence can be translated into logics using syntax and semantics.
- **Syntax** - Syntaxes are the rules which decide how we can construct legal sentences in the logic.
 - It determines which symbol we can use in knowledge representation.
 - How to write those symbols.
- **Semantics** - Semantics are the rules by which we can interpret the sentence in the logic.
 - Semantic also involves assigning a meaning to each sentence.
 - It can be two logics : Propositional Logics and Predicate logics
- Advantages of logical representation:
 - It enables us to do logical reasoning.
 - It is the basis for the programming languages.
- Disadvantages of logical Representation:
 - It has some restrictions and is challenging to work with.
 - It may not be very natural, and inference may not be so efficient.

2. Semantic Network Representation

- Semantic networks are alternative of predicate logic for knowledge representation.
- In Semantic networks, we can represent our knowledge in the form of graphical networks.
- Nodes to representing objects and arcs to represent relationship between those objects.
- Semantic networks can categorize the object in different forms and can also link those objects.
- show the connectivity of one object with another object
- Are easy to understand and can be easily extended.
- It consist of mainly two types of relations:
 - IS-A relation (Inheritance)
 - Kind-of-relation

Drawbacks in Semantic representation:

- Takes more computational time at runtime as we need to traverse the complete network tree to answer some questions.
- It might be possible in the worst-case scenario that after traversing the entire tree, we find that the solution does not exist in this network.
- Semantic networks try to model human-like memory to store the information, but in practice, it is not possible to build such a vast semantic network.
- It is inadequate as they do not have any equivalent quantifier, e.g., for all, for some, none, etc.
- Do not have any standard definition for the link names.

Advantages of Semantic network:

- It is a natural representation of knowledge.

- It conveys meaning in a transparent manner.
- It is simple and easily understandable.

3. **Frame Representation**

- A frame is a record like structure which consists of a collection of attributes and its values to describe an entity in the world. Frames are the AI data structure which divides knowledge into substructures by representing stereotypes situations. It consists of a collection of slots and slot values. These slots may be of any type and sizes.
- Slots have names and values which are called facets.
 - Facets: The various aspects of a slot is known as Facets. Facets are features of frames which enable us to put constraints on the frames.
- A frame may consist of any number of slots, and a slot may include any number of facets and facets may have any number of values.
- A frame is also known as slot-filler knowledge representation in artificial intelligence.

Advantages of frame representation:

- It makes the programming easier by grouping the related data.
- It is comparably flexible and used by many applications in AI.
- It is very easy to add slots for new attribute and relations.
- It is easy to include default data and to search for missing values.
- Frame representation is easy to understand and visualize.

Disadvantages of frame representation:

- In frame system inference mechanism is not be easily processed.
- Inference mechanism cannot be smoothly proceeded by frame representation.

4. **Production Rules**

- In production rules agent checks for the condition and if the condition exists then production rule fires and corresponding action is carried out.
 - The condition part of the rule determines which rule may be applied to a problem.
 - And the action part carries out the associated problem-solving steps. This complete process is called a recognize-act cycle.
- The working memory contains the description of the current state of problems-solving and rule can write knowledge to the working memory. This knowledge match and may fire other rules.

- If there is a new situation (state) generates, then multiple production rules will be fired together, this is called conflict set. In this situation, the agent needs to select a rule from these sets, and it is called a conflict resolution.
- Advantages of Production rule:
 - The production rules are expressed in natural language.
 - The production rules are highly modular, so we can easily remove, add or modify an individual rule.
- Disadvantages of Production rule:
 - Production rule system does not exhibit any learning capabilities, as it does not store the result of the problem for the future uses.
 - During the execution of the program, many rules may be active hence rule-based production systems are inefficient.

4.7 Propositional Logic

Propositional logic (PL) is the simplest form of logic where all the statements are made by propositions. A proposition is a declarative statement which is either true or false. It is a technique of knowledge representation in logical and mathematical form.

Following are some basic facts about PL:

- Propositional/Boolean logic as it works on 0 and 1.
- In PL, we use symbolic variables to represent the logic, and we can use any symbol for a representing a proposition, such A, B, C, P, Q, R, etc.
- Propositions can be either **true or false**, but it cannot be both.
- Propositional logic consists of an **object, relations or function, and logical connectives**.
- These connectives are also called logical operators.
- The propositions and connectives are the basic elements of the propositional logic.
- Connectives can be said as a logical operator which connects two sentences.
- A proposition formula which is always true is called **tautology**, and it is also called a valid sentence.
- A proposition formula which is always false is called **Contradiction**.
- A proposition formula which has both true and false values is called **Contingency**.
- Statements which are questions, commands, or opinions are not propositions such as "Where is Rohini", "How are you", "What is your name", are not propositions.

Syntax of propositional logic:

The syntax of propositional logic defines the allowable sentences for the knowledge representation.

There are two types of Propositions:

- Atomic Proposition - are the simple propositions consists of a single proposition symbol.

- These are the sentences which must be either true or false.
- $2+2$ is 4, it is an atomic proposition as it is a true fact.
- "The Sun is cold" is also a proposition as it is a false fact.
- Compound proposition - are constructed by combining simpler or atomic propositions, using parenthesis and logical connectives.
 - a) "It is raining today, and street is wet."
 - b) "Ankit is a doctor, and his clinic is in Mumbai."

4.7.1 Logical connectives in PL

Logical connectives are used to connect **two simpler propositions** or representing a sentence logically.

There are mainly five connectives

- Negation: A sentence such as $\neg P$ is called negation of P. A literal can be either Positive literal or negative literal.
- Conjunction: A sentence which has $\wedge$ connective such as, $P \wedge Q$ is called a conjunction.
 - Example: Rohan is intelligent and hardworking. It can be written as,
 - P = Rohan is intelligent,
 - Q = Rohan is hardworking. $\rightarrow P \wedge Q$.
- Disjunction: A sentence which has $\vee$ connective, such as $P \vee Q$. is called disjunction, where P and Q are the propositions.
 - Example: "Ritika is a doctor or Engineer",
 - Here P = Ritika is Doctor. Q = Ritika is Doctor, so we can write it as $P \vee Q$.
- Implication: A sentence such as $P \rightarrow Q$, is called an implication. Implications are also known as if-then rules.
 - If it is raining, then the street is wet.
 - Let P = It is raining, and Q = Street is wet, so it is represented as $P \rightarrow Q$
- Biconditional: A sentence such as $P \Leftrightarrow Q$ is a Biconditional sentence, example If I am breathing, then I am alive
 - P = I am breathing, Q = I am alive, it can be represented as $P \Leftrightarrow Q$.

Precedence of connectives - Just like arithmetic operators, there is a precedence order for propositional connectors or logical operators.

- parenthesis $\rightarrow$ negation $\rightarrow$ conjunction $\rightarrow$ dis-junction $\rightarrow$ Implication $\rightarrow$ bi-implication

Logical equivalence – it is one of the features of PL which means two propositions are said to be logically equivalent if and only if the columns in the truth table are identical to each other.

Properties of Operators:

Commutatively:

$$P \wedge Q = Q \wedge P, \text{ or}$$

$$P \vee Q = Q \vee P.$$

Associativity:

$$(P \wedge Q) \wedge R = P \wedge (Q \wedge R),$$

$$(P \vee Q) \vee R = P \vee (Q \vee R)$$

Identity element:

$$P \wedge \text{True} = P,$$

$$P \vee \text{True} = \text{True}.$$

Distributive:

$$P \wedge (Q \vee R) = (P \wedge Q) \vee (P \wedge R).$$

$$P \vee (Q \wedge R) = (P \vee Q) \wedge (P \vee R).$$

DE Morgan's Law:

$$\neg (P \wedge Q) = (\neg P) \vee (\neg Q)$$

$$\neg (P \vee Q) = (\neg P) \wedge (\neg Q).$$

Double-negation elimination:

$$\neg (\neg P) = P.$$

Limitations of Propositional logic:

- We cannot represent relations like **ALL, some, or none** with propositional logic. Example:
 - All the girls are intelligent.
 - Some apples are sweet.
- Propositional logic has **limited expressive power**.
- In propositional logic, we **cannot describe statements in terms of their properties or logical relationships**.

4.7.2 Inference in Propositional Logic

- **Create new logic from old logic or by evidence**, so generating the conclusions from evidence and facts is termed as **Inference**.
- Inference rules: Inference rules are the templates for generating valid arguments.
 - Inference rules are applied to derive proofs in artificial intelligence, and
 - The proof is a sequence of the conclusion that leads to the desired goal.

In inference rules, the implication among all the connectives plays an important role.

- Implication: It is one of the logical connectives which can be represented as $P \rightarrow Q$.
- Converse: The converse of implication, which means the right-hand side proposition goes to the left-hand side and vice-versa. It can be written as $Q \rightarrow P$.
- Contrapositive: The negation of converse is termed as contrapositive, and it can be represented as $\neg Q \rightarrow \neg P$.
- Inverse: The negation of implication is called inverse. It can be represented as $\neg P \rightarrow \neg Q$.

1. **Modus Ponens** - The Modus Ponens rule is one of the most important rules of inference, and it states that **if P and $P \rightarrow Q$ is true**, then we can infer that Q will be true. It can be represented as:

2. **Modus Tollens** - The Modus Tollens rule state that **if $P \rightarrow Q$ is true and $\neg Q$ is true, then $\neg P$ will also true.**
3. **Hypothetical Syllogism** - The Hypothetical Syllogism rule state that **if $P \rightarrow R$ is true whenever $P \rightarrow Q$ is true and $Q \rightarrow R$ is true.**
4. **Disjunctive Syllogism** - The Disjunctive syllogism rule state that **if $P \vee Q$ is true, and $\neg P$ is true, then Q will be true.**
5. **Addition** - The Addition rule is one the common inference rule, and it states that **If P is true, then $P \vee Q$ will be true.**
6. **Simplification** - The simplification rule state **that if $P \wedge Q$ is true, then Q or P will also be true.**
7. **Resolution** - The Resolution rule state **that if $P \vee Q$ and $\neg P \wedge R$ is true, then $Q \vee R$ will also be true.**

4.8 Predicate (First-Order) Logic

FOL is an extension to propositional logic and it sufficiently expressive to represent the natural language statements in a concise way. First-order logic is a powerful language that develops information about the objects in a more easy way and can also express the relationship between those objects.

- First-order logic (like natural language) does not only assume that the world contains facts like propositional logic but also assumes the following things in the world:
 - **Objects:** A, B, people, numbers, colors, wars, theories, squares..
 - **Relations:** It can be unary relation such as: red, round, is adjacent, or n-any relation such as: the sister of, brother of, has color, comes between
 - **Function:** Father of, best friend, end of,
- **Syntax** has to do with what ‘things’ (symbols, notations) one is allowed to use in the language and in what way.
 - Alphabet, Language constructs, Sentences to assert knowledge
- **Semantics:** Formal meaning, which has to do what those sentences with the alphabet and constructs are supposed to mean.
- Following are the basic elements of **FOL syntax**:
- **Atomic sentences** - are the most basic sentences of FOL and are formed from a predicate symbol followed by a parenthesis with a sequence of terms.
 - We can represent atomic sentences as **Predicate (term1, term2,, term n).**
 - Example: Ravi and Ajay are brothers: $\Rightarrow$ Brothers(Ravi, Ajay).
 - Chinky is a cat: $\Rightarrow$ cat (Chinky).
- **Complex Sentences** - are made by combining atomic sentences using connectives.
- FOL statements can be divided into two parts:

- **Subject:** is the main part of the statement.
- **Predicate:** is a relation, which binds two atoms together in a statement.

4.8.1 Quantifiers in FOL

- A quantifier is a language element which generates quantification, and quantification specifies the **quantity** of specimen in the universe of discourse.
- These are the symbols that permit to determine or identify the range and scope of the variable in the logical expression.
- There are two types of quantifier:
 - **Universal Quantifier, (for all, everyone, everything)**
 - **Existential quantifier, (for some, at least one).**

1. **Universal Quantifier:** - is a symbol of logical representation, which specifies that the statement within its **range is true for everything or every instance of a particular thing.**

- It is represented by a symbol $\forall$, which resembles an inverted A.
- In universal quantifier we use implication " $\rightarrow$ ".
- If x is a variable, then $\forall x$ is read as:
- For all x , For each x, For every x.

Example:

- All man drink coffee.
- Let a variable x which refers to a cat so all x can be represented in UOD as below:
- $\forall x \text{ man}(x) \rightarrow \text{drink}(x, \text{coffee})$.
- It will be read as: There are all x where x is a man who drink coffee.

2. **Existential Quantifier:** - are the type of quantifiers, which express that the statement within its scope is **true for at least one instance of something.**

- It is denoted by the logical operator $\exists$, which resembles as inverted E.
- In Existential quantifier we **always use AND or Conjunction symbol ($\wedge$).**
- If x is a variable, then existential quantifier will be $\exists x$ or $\exists(x)$.
- And it will be read as:
- There exists a 'x.' , For some 'x.', For at least one 'x.'
- Example:
 - Some boys are intelligent.
 - $\exists x: \text{boys}(x) \wedge \text{intelligent}(x)$
 - It will be read as: There are some x where x is a boy who is intelligent.

NB: The main connective for **universal quantifier $\forall$ is implication $\rightarrow$ and** The main connective for **existential quantifier $\exists$ is and $\wedge$.**

Properties of Quantifiers:

- In universal quantifier, $\forall x \forall y$ is similar to $\forall y \forall x$.
- In Existential quantifier, $\exists x \exists y$ is similar to $\exists y \exists x$.
- $\exists x \forall y$ is not similar to $\forall y \exists x$.

Some Examples of FOL using quantifier:

- All birds fly.
 - The predicate is fly(bird)
 - $\forall x \text{ bird}(x) \rightarrow \text{fly}(x)$.
- Every man respects his parent.
 - The predicate is respects(X, Y), x = man, y = parent
 - $\forall x \text{ man}(x) \rightarrow \text{respects}(x, \text{parent})$.
- Some boys play cricket.
 - the predicate is "play(x, y)," where x= boys, and y= game.
 - $\exists x \text{ boys}(x) \rightarrow \text{play}(x, \text{cricket})$.
- Not all students like both Mathematics and Science.
 - the predicate is "like(x, y)," where x= student, and y= subject.
 - $\neg \forall (x) [\text{student}(x) \rightarrow \text{like}(x, \text{Mathematics}) \wedge \text{like}(x, \text{Science})]$.
- The quantifiers interact with variables which appear in a suitable ways.
- Two types of variables in FOL
 - **Free Variable:** A variable is said to be a free variable in a formula if it occurs outside the scope of the quantifier. Example: $\forall x \exists (y)[P(x, y, z)]$, where z is a free variable.
 - **Bound Variable:** A variable is said to be a bound variable in a formula if it occurs within the scope of the quantifier. Example: $\forall x [A(x) B(y)]$, here x and y are the bound variables.

4.8.2 Inference in First-Order Logic

- Inference in First-Order Logic is used to deduce new facts or sentences from existing sentences.
- Basic terminologies used in FOL.
- Substitution - is a fundamental operation performed on terms and formulas.
 - It occurs in all inference systems in first-order logic.
 - The substitution is complex in the presence of quantifiers in FOL.
 - If we write $F[a/x]$, so it refers to substitute a constant "a" in place of variable "x".
 - **FOL** is capable of expressing facts about some or all objects in the universe.
- Equality-FOL does not only use **predicate and terms** for making atomic sentences but also uses **equality** (two terms refer to the same object).
 - Example: Brother (John) = Smith.

- As in the above example, the object referred by the Brother (John) is similar to the object referred by Smith. The equality symbol can also be used with negation to represent that two terms are not the same objects.
 - Example: $\neg(x=y)$ which is equivalent to $x \neq y$.

1. Universal Generalization:

Universal generalization is a valid inference rule which states that if premise $P(c)$ is true for any arbitrary element c in the universe of discourse, then we can have a conclusion as $\forall x P(x)$.

- Example: $P(c)$: "A byte contains 8 bits", so for $\forall x P(x)$ "All bytes contain 8 bits.", it will also be true.

2. Universal Instantiation:

- The UI rule state that we can infer any sentence $P(c)$ by substituting a ground term c (a constant within domain x) from $\forall x P(x)$ for any object in the universe of discourse.
- Universal instantiation/universal elimination is a valid inference rule.
- It can be applied multiple times to add new sentences.
- The new KB is logically equivalent to the previous KB.
- As per UI, we can infer any sentence obtained by substituting a ground term for the variable.

Example:1.

- IF "Every person like ice-cream" $\Rightarrow \forall x P(x)$ so we can infer that
- "John likes ice-cream" $\Rightarrow P(c)$

3. Existential Instantiation:

- Existential instantiation/Existential Elimination, which is a valid inference rule in first-order logic.
 - It can be applied only once to replace the existential sentence.
 - The new KB is not logically equivalent to old KB, but it will be satisfiable if old KB was satisfiable.
 - This rule states that one can infer $P(c)$ from the formula given in the form of $\exists x P(x)$ for a new constant symbol c .
 - The restriction with this rule is that c used in the rule must be a new term for which $P(c)$ is true.

Example:

- From the given sentence: $\exists x \text{Crown}(x) \wedge \text{OnHead}(x, \text{John})$,
- So we can infer: $\text{Crown}(K) \wedge \text{OnHead}(K, \text{John})$, as long as K does not appear in the knowledge base.
- The above used K is a constant symbol, which is called Skolem constant.
- The Existential instantiation is a special case of Skolemization process.

4. Existential introduction:

- Existential introduction/existential generalization is a valid inference rule in FOL and it is stated that
 - If there is some elements C in the universe of discourse which has a property P, then we can infer that there exists something in the universe which has the property P

Example: Let's say that,

- "Priyanka got good marks in English."
- "Therefore, someone got good marks in English."
- It can be represented as

4.8.3 Unification in FOL

- **Unification** is a process of making two different logical atomic expressions identical by finding a **substitution**. It depends on the **substitution process**.
- It takes two literals as input and makes them identical using **substitution**.
- Let Ψ_1 and Ψ_2 be two atomic sentences and σ be a unifier such that, $\Psi_1\sigma = \Psi_2\sigma$, then it can be expressed as UNIFY(Ψ_1 , Ψ_2).
- Example: Find the MGU for Unify{King(x), King(John)}
- Let $\Psi_1 = \text{King}(x)$, $\Psi_2 = \text{King}(\text{John})$,
- **Substitution** $\theta = \{\text{John}/x\}$ is a unifier for these atoms and applying this substitution, and both expressions will be identical.
- **The UNIFY algorithm** is used for unification, which takes two atomic sentences and returns a unifier for those sentences (If any exist).
- Unification is a key component of all first-order inference algorithms.
- The substitution variables are called **Most General Unifier or MGU**.
- E.g. Let's say there are two different expressions, **P(x, y)**, and **P(a, f(z))**.
- $P(x, y) \dots\dots\dots (i)$
- $P(a, f(z)) \dots\dots\dots (ii)$
- Substitute **x** with **a**, and **y** with **f(z)** in the first expression, and it will be represented as **a/x and f(z)/y**.
- With both the substitutions, the first expression will be identical to the second expression and the substitution set will be: **[a/x, f(z)/y]**.
- **Conditions for Unification:**
 - Predicate symbol must be same, atoms or expression with different predicate symbol can never be unified.
 - Number of Arguments in both expressions must be identical.
 - It will fail if there are two similar variables present in the same expression.

4.8.4 Resolution in FOL

- **Resolution** - is a theorem proving technique that proceeds by building **refutation proofs**, i.e., **proofs by contradictions**.
- **Resolution** is used, if there are various statements are given, and we need to prove a conclusion of those statements.
- **Unification** is a key concept in proofs by resolutions.
- Resolution is a single inference rule which can efficiently operate on the **conjunctive normal form** or **clausal form**.
- **Clause (a unit clause)** - Disjunction of literals (an atomic sentence).
- **Conjunctive Normal Form**: A sentence represented as a **conjunction** of clauses.

Steps for Resolution:

1. Conversion of facts into first-order logic.
2. Convert FOL statements into CNF
3. Negate the statement which needs to prove (proof by contradiction)
4. Draw resolution graph (unification).

4.9 Reasoning

- Reasoning - is the mental process of deriving logical conclusion and making predictions from available knowledge, facts, and beliefs. "Reasoning is a way to infer facts from existing data." It is a general process of thinking rationally, to find valid conclusions. In artificial intelligence, the reasoning is essential so that the machine can also think rationally as a human brain, and can perform like a human.
- A. **Deductive reasoning** - it deducing new information from logically related known information.
- It is the form of valid reasoning, which means the argument's conclusion must be true when the premises are true.
 - Deductive reasoning is a type of propositional logic in AI, and it requires various rules and facts.
 - It is a top-down reasoning, and contradictory to inductive reasoning.
 - The truth of the premises guarantees the truth of the conclusion.
 - Deductive reasoning **mostly starts from the general premises** to the **specific conclusion**.
- B. **Inductive reasoning**: is a form of reasoning to arrive at a conclusion using limited sets of facts by the process of generalization.
- It starts with the **series of specific facts or data** and reaches to a **general statement or conclusion**.

- Inductive reasoning is a type of propositional logic, which is also known as cause-effect reasoning or bottom-up reasoning.
- We use historical data or various premises to generate a generic rule, for which premises support the conclusion.

C. **Abductive reasoning:** It is a form of logical reasoning which starts with single or multiple observations then seeks to find the most likely **explanation or conclusion** for the observation.

- It is an extension of deductive reasoning, but in abductive reasoning, **the premises do not guarantee** the conclusion.
- Example:
 - Implication: Cricket ground is wet if it is raining
 - Axiom: Cricket ground is wet.
 - Conclusion It is raining.

D. Common Sense Reasoning

- It is an informal form of reasoning, which can be gained through experiences.
- It simulates the human ability to make presumptions about events which occurs on every day.
- It relies on good judgment rather than exact logic and operates on heuristic knowledge and heuristic rules.

Example:

- One person can be at one place at a time.
- If I put my hand in a fire, then it will burn.
- Human mind can easily understand and assume.

E. Monotonic Reasoning:

- In monotonic reasoning, once the conclusion is taken, then it will remain the same even if we add some other information to existing information in our knowledge base.
- In monotonic reasoning, adding knowledge does not decrease the set of prepositions that can be derived.
- To solve monotonic problems, we can derive the valid conclusion from the available facts only, and it will not be affected by new facts.
- Monotonic reasoning is not useful for the real-time systems, as in real time, facts get changed, so we cannot use monotonic reasoning.
- Monotonic reasoning is used in conventional reasoning systems, and a logic-based system is monotonic.
 - Any theorem proving is an example of monotonic reasoning.

F. Non-monotonic Reasoning

- Some conclusions may be invalidated if we add some more information to **KB**.
- Logic will be said as non-monotonic if some conclusions can be invalidated by adding more knowledge into our knowledge base.
- Non-monotonic reasoning deals with **incomplete and uncertain models**.
- "Human perceptions for various things in daily life, "is a general example
- Example: - **Birds can fly**
 - **Penguins cannot fly**
 - **Pitty is a bird**
- So from the above sentences, we can conclude that **Pitty can fly**.
- if we add one another sentence into knowledge base "Pitty is a penguin", which concludes "**Pitty cannot fly**", so it invalidates the above conclusion.

4.10 Reasoning under uncertainty

- Certainty - propositional logic and predicate(first order logic).
- Sure about predicates
- **Uncertainty** – probability reasoning
 - Unsure about predicates
 - $A \rightarrow B$, which means if A is true then B is true, but consider a situation where we are not sure about whether A is true or not then we cannot express this statement, this situation is called **uncertainty**.
 - causes of uncertainty
 - **Information occurred from unreliable sources.**
 - **Experimental Errors**
 - **Equipment fault**
 - **Temperature variation**
 - **Climate change.**
- **Probabilistic reasoning** is a way of knowledge representation where we apply the concept of probability to indicate the uncertainty in knowledge.
- In probabilistic reasoning, we combine probability theory with logic to handle the uncertainty.
- Example – we are not sure whether it will happen or not
 - "It will rain today,"
 - "behavior of someone for some situations,"
 - "A match between two teams or two players."
- Needs of probability reasoning

- When there are unpredictable outcomes.
- When specifications or possibilities of predicates becomes too large to handle.
- When an unknown error occurs during an experiment.
- In probabilistic reasoning, there are two ways to solve problems with uncertain knowledge:
 - Bayes' rule and Bayesian Statistics
- **Probability** - a chance that an uncertain event will occur.
 - It is the numerical measure of the likelihood that an event will occur.
 - The value of probability always remains between 0 and 1 that represent ideal uncertainties.
 - $0 \leq P(A) \leq 1$, where $P(A)$ is the probability of an event A .
 - $P(A) = 0$, indicates total uncertainty in an event A .
 - $P(A) = 1$, indicates total certainty in an event A .
 - $P(\neg A)$ = probability of a not happening event.
 - $P(\neg A) + P(A) = 1$.
- To find the probability of an uncertain event we use the formula
- **Probability of occurrence** = $\frac{\text{number of desired outcome}}{\text{total number of outcome}}$
- **Event** - Each possible outcome of a variable.
- **Sample space** - The collection of all possible events.
- **Random variables**: are used to represent the events and objects in the real world.
- **Prior probability**: The prior probability of an event is probability computed before observing new information.
- **Posterior Probability**: The probability that is calculated after all evidence or information has taken into account. It is a combination of prior probability and new information.
- **Conditional probability** is a probability of occurring an event when another event has already happened.
 - Let's suppose, we want to calculate the event A when event B has already occurred, "the probability of A under the conditions of B ", it can be written as: $P(A/B) = P(A \cap B)/P(B)$
- Where $P(A \cap B)$ = Joint probability of A and B
- $P(B)$ = Marginal probability of B .
 - If the probability of A is given and we need to find the probability of B , then it will be given as:

$$P(B/A) = P(A \cap B)/P(A)$$
- Example:
- In a class, there are 70% of the students who like English and 40% of the students who likes English and mathematics, and then what is the percent of students those who like English also like mathematics?

- Solution:
 - Let, A is an event that a student likes Mathematics
 - B is an event that a student likes English.
 - $P(A/B) = P(A \cap B)/P(B) = 0.4/0.7 = 57\%$
 - Hence, 57% are the students who like English also like Mathematics.
- **Bayes' theorem or Bayes' rule, Bayes' law, or Bayesian reasoning**, which determines the probability of an event with uncertain knowledge.
- It is a way to calculate the value of $P(B|A)$ with the knowledge of $P(A|B)$.
- Bayes' theorem allows updating the probability prediction of an event by observing new information of the real world.
- Bayes' theorem can be derived using product rule and conditional probability of event A with known event B:
 - As from product rule we can write:
 - $P(A \cap B) = P(A|B) P(B)$ or
 - Similarly, the probability of event B with known event A:
 - $P(A \cap B) = P(B|A) P(A)$
- Bayes' theorem
 - Equating right hand side of both the equations, we will get:
 - $P(A|B) = \frac{P(B|A)P(A)}{P(B)}$ (a) - Bayes' rule or Bayes' theorem.
 - It shows the simple relationship between joint and conditional probabilities. Here,
 - $P(A|B)$ is known as **posterior**, which we need to calculate, and it will be read as Probability of hypothesis A when we have occurred an evidence B.
 - $P(B|A)$ is called the **likelihood**, in which we consider that hypothesis is true, then we calculate the probability of evidence.
 - $P(A)$ is called the **prior probability**, probability of hypothesis before considering the evidence
 - $P(B)$ is called **marginal probability**, pure probability of an evidence.
- In the equation (a), in general, we can write $P(B) = P(A) * P(B|A_i)$, hence the Bayes' rule can be written as: $P(A_i|B) = \frac{P(B|A_i)P(A_i)}{\sum P(A_i)P(B|A_i)}$, Where $A_1, A_2, A_3, \dots, A_n$ is a set of mutually exclusive and exhaustive events.
- Example-1:
 - Question: what is the probability that a patient has diseases meningitis with a stiff neck? Assume
 - A doctor is aware that disease meningitis causes a patient to have a stiff neck, and it occurs 80% of the time. He is also aware of some more facts, which are given as follows:

- The Known probability that a patient has meningitis disease is 1/30,000.
- The Known probability that a patient has a stiff neck is 2%.
- Let **a** be the proposition that patient has stiff neck and **b** be the proposition that patient has meningitis. , so we can calculate the following as:
 - **$P(a|b) = 0.8$, $P(b) = 1/30000$, $P(a) = .02$**
 - **$P(b/a) = (p(a/b)*p(b))/p(a) = (0.8*1/3000)/0.02 = 0.0013333333$.**
 - Hence, we can assume that 1 patient out of 750 patients has meningitis disease with a stiff neck.
- Example-2:
- Question: From a standard deck of playing cards, a single card is drawn. The probability that the card is king is 4/52, then calculate posterior probability $P(\text{King}|\text{Face})$, which means the drawn face card is a king card.
- Solution:
- $$P(\text{king}/\text{face}) = \frac{P(\frac{\text{face}}{\text{king}})*P(\text{king})}{P(\text{face})} \dots\dots\dots i$$
 - $P(\text{king})$: probability that the card is King= 4/52= 1/13
 - $P(\text{face})$: probability that a card is a face card= 3/13
 - $P(\text{Face}|\text{King})$: probability of face card when we assume it is a king = 1
 - Putting all values in equation (i) we will get:
 - $P(\text{king}/\text{face}) = \frac{(\frac{1}{13})*1}{3/13} = 1/3$ is the probability that a face card is a king card.

4.11 Summery

4.12 Sample Questions

- The process of capturing the inference process as Single Inference Rule is known as_____**

A) Generalized Modus Ponens	C) Variables
B) Ponens	D) Clause
- Which algorithm takes two sentences as input and returns a Unifier?**

A) Inference	C) Hill-Climbing
B) Unify algorithm	D) Depth-first search
- First order logic Statements contains_____**

A) Predicate and Subject	C) Predicate and Subject
B) Predicate and Preposition	D) Subject and an Object
- The Bayesian Network gives_____**

A) A complete description of the domain
B) Partial Description of the domain

C) A complete description of the domain

D) None of these

5. The probabilistic reasoning depends upon_____

A) Observations

C) Likelihood

B) Estimation

D) All of these

6. Hybrid Bayesian Network consist_____.

A) both discrete and continuous variable

C) Continuous Variable only

B) Discrete variables only

D) Discontinuous Variable

7. How do you represent “All dogs have tails”?

A) $\forall x: \text{dog}(x) \rightarrow \text{tail}(x)$

C) $\forall x: \text{dog}(y) \rightarrow \text{tail}(x)$

B) $\forall x: \text{dog}(x) \rightarrow \text{tail}(y)$

D) $\forall x: \text{dog}(x) \rightarrow \text{tail}(x)$

8. A production rule consists of

A) A set of Rule

B) A sequence of steps

C) Set of Rule & sequence of steps

D) Arbitrary representation to problem

9. _____ is used to demonstrate, on a purely syntactic basis, that one formula is a logical consequence of another formula.

A) Deductive Systems

B) Inductive Systems

C) Reasoning with Knowledge Based Systems

D) Search Based Systems

10. What is the condition of variables in first-order literals?

A) Existentially quantified

B) Universally quantified

C) Both Existentially & Universally quantified

D) None of the mentioned

Chapter Five: Expert System

Objectives

- Introduction
- Applications of Expert Systems
- Expert Systems Technologies
- Benefits of Expert Systems
- Expert System Limitations
- The Architecture of Expert Systems
- Components of Expert Systems
- The Knowledge bases
- The Inference Engine
- The User Interface
- Development of Expert System

5.1 Introduction

The expert systems are the computer applications developed to solve complex problems in a particular domain, at the level of extra-ordinary human intelligence and expertise. **Some characteristics of expert system are High performance, Understandable, Reliable, and Highly responsive.**

Capabilities of Expert Systems include

- Advising
- Instructing and assisting human in decision making
- Demonstrating
- Deriving a solution
- Diagnosing and Explaining
- Interpreting input
- Predicting results
- Justifying the conclusion
- Suggesting alternative options to a problem

Expert Systems are incapable of

- Substituting human decision makers
- Possessing human capabilities
- Producing accurate output for inadequate knowledge base
- Refining their own knowledge

5.2 Applications of Expert Systems

- **Design Domain** - Camera lens design, automobile design.
- **Medical Domain** - Diagnosis Systems to deduce cause of disease from observed data, conduction medical operations on humans.

- **Monitoring Systems** - Comparing data continuously with observed system or with prescribed behaviour such as leakage monitoring in long petroleum pipeline.
- **Process Control Systems** - Controlling a physical process based on monitoring.
- **Knowledge Domain** - Finding out faults in vehicles, computers.
- **Finance/Commerce** - Detection of possible fraud, suspicious transactions, stock market trading, Airline scheduling, cargo scheduling.

5.3 Expert Systems Technologies

- **Expert System Development Environment** – The ES development environment includes hardware and tools. They are –
 - Workstations, minicomputers, mainframes.
 - High level Symbolic Programming Languages such as LISP Programming (LISP) and PROgrammation en LOGique (PROLOG).
 - Large databases.
- **Tools** – They reduce the effort and cost involved in developing an expert system to large extent.
 - Powerful editors and debugging tools with multi-windows.
 - They provide rapid prototyping
 - Have Inbuilt definitions of model, knowledge representation, and inference design
- **Shells** – A shell is nothing but an expert system without knowledge base.
 - A shell provides the developers with knowledge acquisition, inference engine, user interface, and explanation facility.
 - For example
 - **Java Expert System Shell (JESS)** that provides fully developed Java API for creating an expert system. **Vidwan**, a shell developed at the National Centre for Software Technology,
 - **Mumbai** in 1993. It enables knowledge encoding in the form of IF-THEN rules.

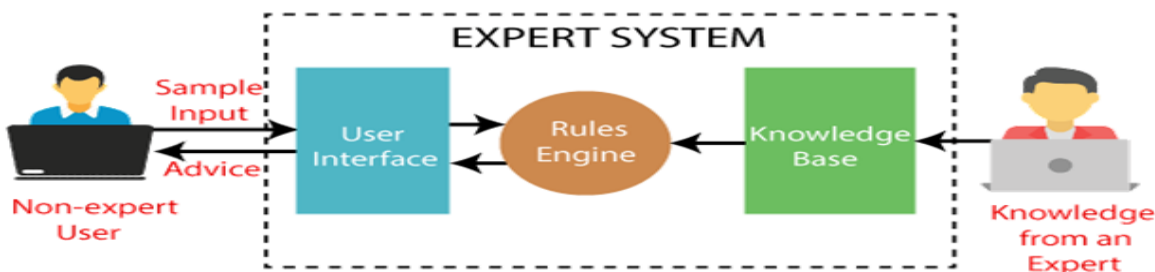
5.4 Benefits of Expert Systems

- **Availability** – are easily available due to mass production of software.
- **Less Production Cost** – Production cost is reasonable. This makes them affordable.
- **Speed** – offer great speed or reduce the amount of work an individual puts in.
- **Less Error Rate** – Error rate is low as compared to human errors.
- **Reducing Risk** – can work in the environment dangerous to humans.
- **Steady response** – work steadily without getting motionless, tensed or fatigued.

5.5 Expert System Limitations

- No technology can offer easy and complete solution.
- Large systems are costly, require significant development time, and computer resources.
- ESs have their limitations which include
 - Limitations of the technology
 - Difficult knowledge acquisition
 - ES are difficult to maintain
 - High development costs

5.6 The Architecture of Expert Systems



5.7 Components of Expert Systems

5.7.1 The Knowledge bases

- It contains domain-specific and high-quality knowledge.
- Knowledge is required to exhibit intelligence.
- The success of any ES majorly depends upon the **collection of highly accurate and precise knowledge**.
- The data is the collection of facts.
- The information is organized as data and facts about the task domain.
- Data, information, and past experience combined together are termed as **knowledge**.
- **Components of Knowledge Base**
- The knowledge base of an ES is a store of both, factual and heuristic knowledge.
 - **Factual Knowledge** – It is the information widely accepted by the Knowledge Engineers and scholars in the task domain.
 - **Heuristic Knowledge** – It is about practice, accurate judgement, one's ability of evaluation, and guessing.
- **Knowledge representation**
 - It is the method used to organize and formalize the knowledge in the knowledge base.

- It is in the form of IF-THEN-ELSE rules.
- **Knowledge Acquisition**
- The success of any expert system majorly depends on the **quality, completeness, and accuracy** of the information stored in the knowledge base.
- The knowledge base is formed by readings from various experts, scholars, and the Knowledge Engineers.
- The knowledge engineer is a person with the qualities of empathy, quick learning, and case analysing skills.
- He acquires information from subject expert by recording, interviewing, and observing him at work, etc. He then categorizes and organizes the information in a meaningful way, in the form of IF-THEN-ELSE rules, to be used by inference machine. The knowledge engineer also monitors the development of the ES

5.7.2 The Inference Engine

- Use of efficient procedures and rules by the Inference Engine is essential in **deducting a correct, flawless solution**.
- In case of knowledge-based ES, the Inference Engine acquires and manipulates the knowledge from the knowledge base to arrive at a particular solution.
- In case of rule-based ES, it –
 - Applies rules repeatedly to the facts, which are obtained from earlier rule application.
 - Adds new knowledge into the knowledge base if required.
 - Resolves rules conflict when multiple rules are applicable to a particular case
- To recommend a solution, the Inference Engine uses
- **Forward Chaining**
 - It is a strategy of an expert system to answer the question, “What can happen next?”
 - Here, the Inference Engine follows the chain of conditions and derivations and finally deduces the outcome.
 - It considers all the facts and rules, and sorts them before concluding to a solution.
 - This strategy is followed for working on conclusion, result, or effect.
 - For example, prediction of share market status as an effect of changes in interest rates.
 - Properties of Forward-Chaining:
 - It is a **down-up approach**, as it moves from bottom to top.
 - It is a process of making a conclusion based on known facts or data, by starting from the initial state and reaches the goal state.

- Forward-chaining approach is also called as **data-driven** as we reach to the goal using available data.
- Forward -chaining approach is commonly used in the expert system, such as CLIPS, business, and production rule systems.
- **Backward Chaining**
 - With this strategy, an expert system finds out the answer to the question, “Why this happened?”
 - On the basis of what has already happened, the Inference Engine tries to find out which conditions could have happened in the past for this result.
 - This strategy is followed for finding out cause or reason.
 - For example, diagnosis of blood cancer in humans
 - **Properties of backward chaining:**
 - Backward-chaining is based on modus ponens inference rule.
 - The goal is broken into sub-goal or sub-goals to prove the facts true.
 - It is called a goal-driven approach, as a list of goals decides which rules are selected and used.
 - It is used in game theory, automated theorem proving tools, inference engines, proof assistants, and various AI applications.
 - It mostly used a depth-first search strategy for proof.

5.7.3 The User Interface

- User interface provides interaction between user of the ES and the ES itself.
- Natural Language Processing so as to be used by the user who is well-versed in the task domain
- It explains how the ES has arrived at a particular recommendation.
- The explanation may appear in the following forms
 - Natural language displayed on screen.
 - Verbal narrations in natural language.
 - Listing of rule numbers displayed on the screen.
- The user interface makes it easy to trace the credibility of the deductions
- Requirements of Efficient ES User Interface
 - It should help users to accomplish their goals in shortest possible way.
 - It should be designed to work for user’s existing or desired work practices.
 - Its technology should be adaptable to user’s requirements; not the other way around.
 - It should make efficient use of user input.

5.8 Development of Expert System

The process of ES development is iterative. Steps are

- **Identify Problem Domain**
 - The problem must be suitable for an expert system to solve it.
 - Find the experts in task domain for the ES project.
 - Establish cost-effectiveness of the system.
- **Design the System**
 - Identify the ES Technology
 - Know and establish the degree of integration with the other systems and databases.
 - Realize how the concepts can represent the domain knowledge best.
- **Develop the Prototype**
 - Acquire domain knowledge from the expert.
 - Represent it in the form of If-THEN-ELSE rules.
- **Test and Refine the Prototype**
 - The knowledge engineer uses sample cases to test the prototype for any deficiencies in performance.
 - End users test the prototypes of the ES.
- **Develop and Complete the ES**
 - Test and ensure the interaction of the ES with all elements of its environment, including
 - Document the ES project well.
 - Train the user to use ES.
 - end users, databases, and other information systems.
- **Maintain the System**
 - Keep the knowledge base up-to-date by regular review and update.
 - Cater for new interfaces with other information systems, as those systems evolve

5.7 Questions

1. Which of the following is not a Characteristics of Expert Systems?

A. Understandable	C. Unreliable
B. Highly responsive	D. High performance
2. Which of the following is not a Capabilities of Expert Systems?

A. Advising	C. Explaining
B. Demonstrating	D. Expanding

3. Which of the following is Capabilities of Expert Systems?
 - A. Possessing human capabilities
 - B. Suggesting alternative options to a problem
 - C. Refining their own knowledge
 - D. Substituting human decision makers
4. Which of the following are Components of Expert Systems?
 - A. Knowledge Base
 - B. Inference Engine
 - C. User Interface
 - D. All of the above
5. Which of the following is incorrect application of Expert System?
 - A. Design Domain
 - B. Monitoring Systems
 - C. Knowledge Domain
 - D. Systems domain
6. Which of the following is incorrect Expert Systems Limitations?
 - A. Limitations of the technology
 - B. Difficult knowledge acquisition
 - C. Easy to maintain
 - D. High development costs
7. A _____ is nothing but an expert system without knowledge base?
 - A. Tools
 - B. shell
 - C. Expert System
 - D. knowledge
8. Which of the following strategies used by Inference Engine?
 - A. Forward Chaining
 - B. Block Chaining
 - C. Stable Chaining
 - D. Both A and B
9. In Expert System, Knowledge Acquisition means,
 - A. How to get required domain knowledge by the expert system
 - B. System maintenance
 - C. System implementation
 - D. None of the mentioned above
10. In expert system Forward Chaining, is a strategy to answer the question, "____".
 - A. What can happen previously?
 - B. What can happen next?
 - C. Both A and B
 - D. All of the mentioned above

Chapter Six: Learning Agents

Objectives

- Introduction
- Define learning agents, machine learning
- Discuss components of learning system.
- Types of machine learning and describe the algorithms for each learning types.
- Discuss about application of machine learning, pros and cons of machine learning.
- Define neural networks and compare with biological neurons
- ANN for linear equations
- Discuss about processing of ANN (Network Topology, Adjustments of Weights or Learning, and Activation Functions)
- Compare and contrast feedforward network and feedbackward network and discuss its type.
- Define activation function, and its type (linear, bi-polar sigmoid, binary sigmoid).
- Application of ANN, Advantages and disadvantages of ANN

6.1 Introduction

Learning denotes changes in a system that enables the system more efficient next time. Learning is an important feature of “intelligent”. Machine learning is the subfield of AI concerned with intelligent systems that learn from experience and examples. It is the computational study of algorithms that improve performance based on experience

Machine learning is particularly attractive in several real-life problems because of the following reasons:

- Some tasks cannot be defined well except by example
- Working environment of machines may not be known at design time
- Explicit knowledge encoding may be difficult and not available
- Environments change over time
- Biological systems learn

Recently, learning is widely used in a number of application areas including,

- Data mining and knowledge discovery
- Speech/image/video (pattern) recognition
- Adaptive control
- Autonomous vehicles/robots
- Decision support systems
- Bioinformatics
- WWW

Formally, a computer program is said to **learn** from experience **E** with respect to some class of tasks **T** and performance measure **P**, if its performance at tasks in **T**, as measured by **P**, improves with experience **E**.

Thus a learning system is characterized by:

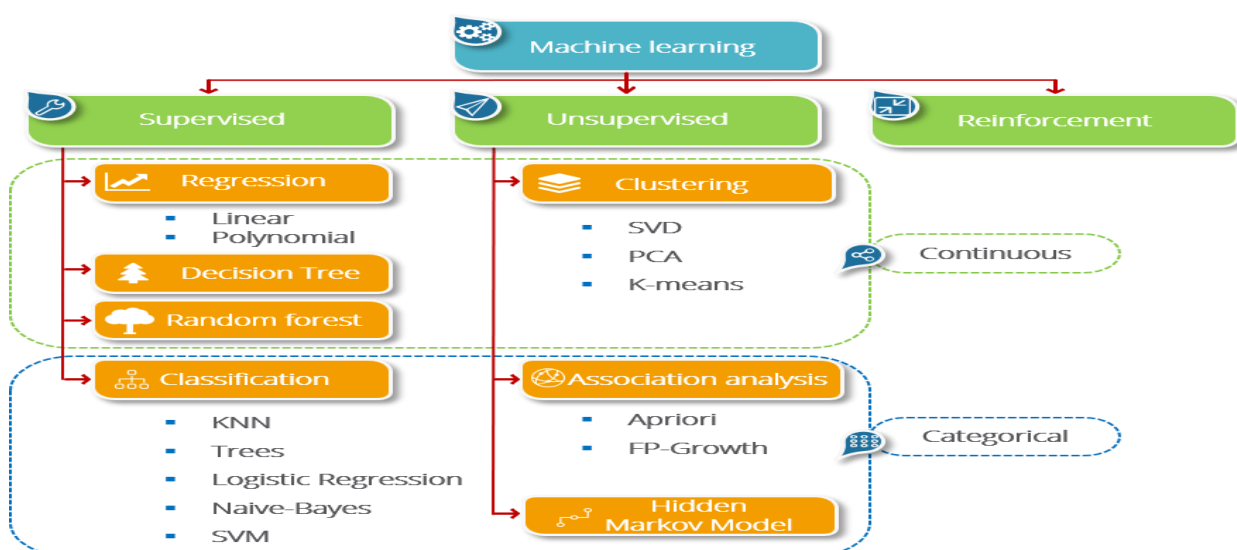
- task **T**
- experience **E**, and
- performance measure **P**

Components of learning system

- **Learning Element** makes changes to the system based on how it's doing
- **Performance Element** is the agent itself that acts in the world
- **Critic** tells the Learning Element how it is doing (e.g., success or failure) by comparing with a fixed standard of performance
- **Problem Generator** suggests "problems" or actions that will generate new examples or experiences that will aid in training the system further

6.2 Types of learning (machine learning)

- Machine learning enables a machine to automatically learn from data, improve performance from experiences, and predict things without being explicitly programmed.
- Want to learn an unknown function $f(x) = y$, where x is an input example and y is the desired output.
- Machine learning constructs or uses the algorithms that learn from historical data.
 - Supervised
 - Unsupervised
 - Reinforcement



⇒ **Supervised learning** implies we are given a set of (x, y) pairs by a "teacher." The system is supplied

with a set of training examples consisting of inputs and corresponding outputs, and is required to discover the relation or mapping between them, e.g. as a series of rules, or a neural network. Agent learns a function from observing example input-output pairs

- Classification
- Regression

⇒ **Unsupervised learning** means we are only given the x 's. In either case, the goal is to estimate f . The system is supplied with a set of training examples consisting only of inputs and is required to discover for itself what appropriate outputs should be,

- Most common task is clustering – e.g. taxi agent notices “bad traffic days”
- Association rule learning
- If someone buys pen and book then he/she has the probability of buying bag
 - Clustering
 - Association

⇒ **Reinforcement learning** is a feedback-based learning method, in which a learning agent gets a reward for each right action and gets a penalty for each wrong action.

The agent learns automatically with these feedbacks and improves its performance. In reinforcement learning, the agent interacts with the environment and explores it. The goal of an agent is to get the most reward points, and hence, it improves its performance. The robotic dog, which automatically learns the movement of his arms, is an example of Reinforcement learning.

A. Decision tree algorithm

⇒ It is a classification and prediction tool having a tree like structure, where each internal node denotes a test on an attribute, each branch represents an outcome of the test, and each leaf node (terminal node) holds a class label. In Business Intelligence, **entropy** is a measure of the randomness in the information being processed. The higher the **entropy**, the harder it is to draw any conclusions from that information.

$$H(X) = - \sum_{i=1}^n P(x_i) \log_b P(x_i)$$

⇒ Information gain can be defined as the amount of information gained about a random variable or signal from observing another random variable. It can be considered as the difference between the entropy of parent node and weighted average entropy of child nodes.

$$IG(S, A) = H(S) - H(S, A)$$

Alternatively,

$$IG(S, A) = H(S) - \sum_{i=0}^{n-1} P(x_i) * H(x_i)$$

⇒ **Gini impurity** is a measure of how often a randomly chosen element from the set would be incorrectly labeled if it was randomly labeled according to the distribution of labels in the subset. Gini impurity is **lower bounded by 0**, with 0 occurring if the data set contains only one class.

$$Gini(E) = 1 - \sum_{j=1}^c p_j^2$$

There are many algorithms there to build a decision tree. They are

CART (Classification and Regression Trees) — This makes use of Gini impurity as metric.

ID3 (Iterative Dichotomiser 3) — This uses entropy and information gain as metric.

B. Regression algorithm

Regression analysis is one of the most commonly used BI techniques in social and behavioural sciences as well as in physical sciences which involves identifying and evaluating the relationship between a dependent variable and one or more independent variables, which are also called predictor or explanatory variables.

- Independent variables are characteristics that can be measured directly; these variables are also called predictor or explanatory variables used to predict or to explain the behavior of the dependent variable.
- Dependent variable is a characteristic whose value depends on the values of independent variables.

- The regression model is represented by the following equation: $y = \beta_0 \pm \beta_1 x_1 \pm \varepsilon_1$

Where

- x independent variable.
- y dependent variable.
- β_1 The Slope of the regression line
- β_0 The intercept point of the regression line and the y axis.
- n Number of cases or individuals.
- $\sum xy$ Sum of the product of dependent and independent variables.
- $\sum x$ = Sum of independent variable.
- $\sum y$ = Sum of dependent variable.
- $\sum x^2$ = Sum of square independent variable.

$$\beta_1 = \frac{n \sum xy - \sum x \sum y}{n \sum x^2 - (\sum x)^2}$$

$$\beta_0 = \bar{y} - \beta_1 \bar{x}$$

6.2 Neural Networks

Artificial neural networks are among the most powerful learning models. They have the versatility to approximate a wide range of complex functions representing multi-dimensional input-output maps. Neural networks also have inherent adaptability, and can perform robustly even in noisy environments. An Artificial Neural Network (ANN) is an information processing paradigm that is inspired by the way biological nervous systems, such as the brain, process information. The key element of this paradigm is the novel structure of the information processing system.

It is composed of large number of highly interconnected simple processing elements (neurons) working in unison to solve specific problems. ANNs, like people, learn by example. An ANN is configured for a specific application, such as pattern recognition or data classification, through a learning process. Learning in biological systems involves adjustments to the synaptic connections that exist between the neurons. This is true of ANNs as well. ANNs can process information at a great speed owing to their highly massive parallelism.

Neural networks, with their remarkable ability to derive meaning from complicated or imprecise data, can be used to extract patterns and detect trends that are too complex to be noticed by either humans or other computer techniques. A trained neural network can be thought of as an "expert" in the category of information it has been given to analyse. This expert can then be used to provide projections given new situations of interest and answer "what if" questions. Other advantages include:

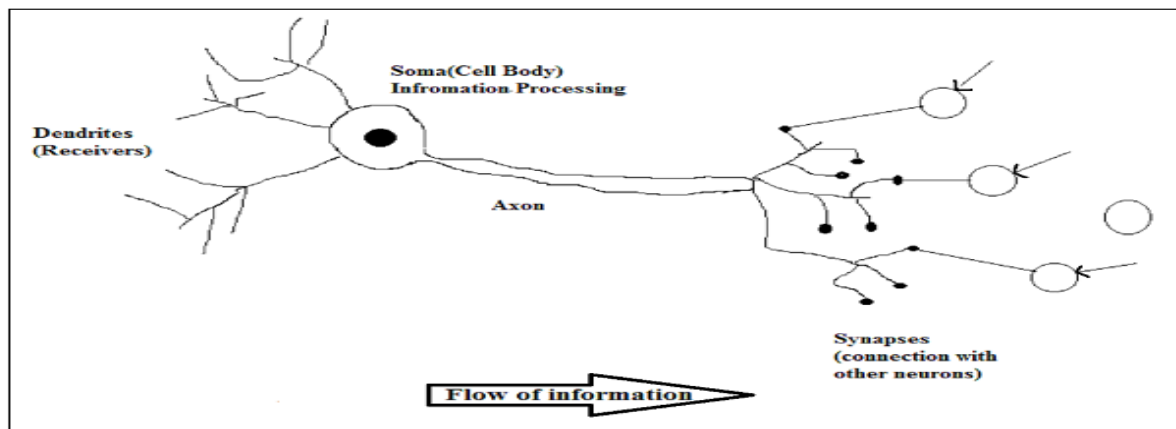
- Adaptive learning: An ability to learn how to do tasks based on the data given for training or initial experience.

- Self-Organisation: An ANN can create its own organisation or representation of the information it receives during learning time.
- Real Time Operation: ANN computations may be carried out in parallel, and special hardware devices are being designed and manufactured which take advantage of this capability.
- Fault Tolerance via Redundant Information Coding: Partial destruction of a network leads to the corresponding degradation of performance. However, some network capabilities may be retained even with major network damage.

6.3 Biological neurons

A nerve cell *neuron* is a special biological cell that processes information. According to an estimation, there are huge number of neurons, approximately 10^{11} with numerous interconnections, approximately 10^{15} . A typical neuron consists of the following four parts

- **Dendrites** – They are tree-like branches, responsible for receiving the information from other neurons it is connected to. In other sense, we can say that they are like the ears of neuron.
- **Soma** – It is the cell body of the neuron and is responsible for processing of information, they have received from dendrites.
- **Axon** – It is just like a cable through which neurons send the information.
- **Synapses** – It is the connection between the axon and other neuron dendrites.



Biological Neural Network <i>BNN</i>	Artificial Neural Network <i>ANN</i>
Soma	Node
Dendrites	Input
Synapse	Weights or Interconnections
Axon	Output

6.4 ANN for linear equations

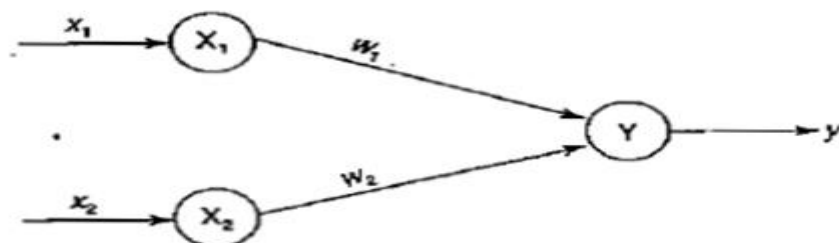
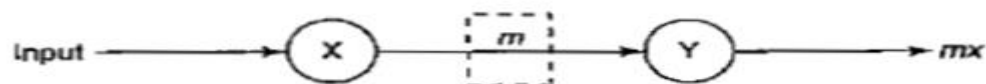


Figure 2-1 Architecture of a simple artificial neuron net.



- To depict the basic operation of a neural net, consider a set of neurons, say X1 and X2, transmitting signals to another neuron, Y. Here X1, and X2 are input neurons, which transmit signals, and Y is the output neuron, which receives signals. Input neurons X1, and X2 are connected to the output neuron Y, over a weighted
- Interconnection links (W1, and W2) as shown in the figure. For the above simple neuron net architecture, the net input has to be calculated in the following way:

$$Y_{in} = X_1W_1 + X_2W_2$$

where x1 and x2 are the activation of the input neurons X1, and X2, i.e., the output of input signals. The output y of the output neuron Y can be obtained applying activation over net input, i.e., the function of the net input:

- $Y = F(y_{in})$
- The function robe applied over the net input is called as activation function.

6.5 Processing of ANN

Processing of ANN depends on the following

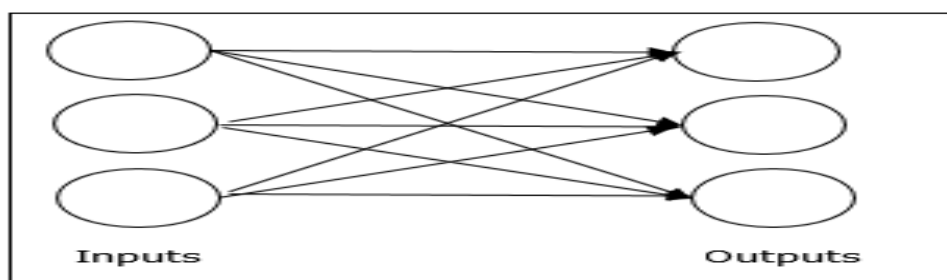
- Network Topology
- Adjustments of Weights or Learning
- Activation Functions

11. A network topology

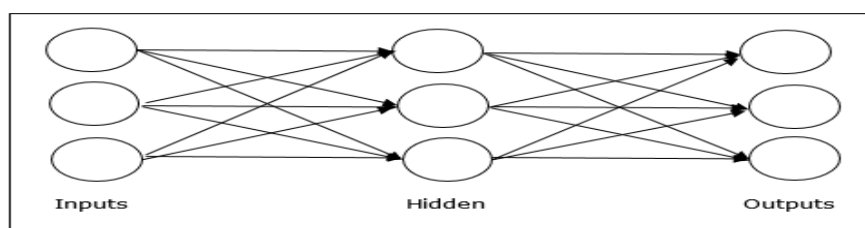
It is the arrangement of a network along with its nodes and connecting lines.

- **Feedforward Network** - It is a non-recurrent network having processing units/nodes in layers and all the nodes in a layer are connected with the nodes of the previous layers. The connection has different weights upon them. There is no feedback loop means the signal can only flow in one direction, from input to output. It may be divided into the following two types –

A. Single layer feedforward network – The concept is of feedforward ANN having only one weighted layer. In other words, we can say the input layer is fully connected to the output layer.

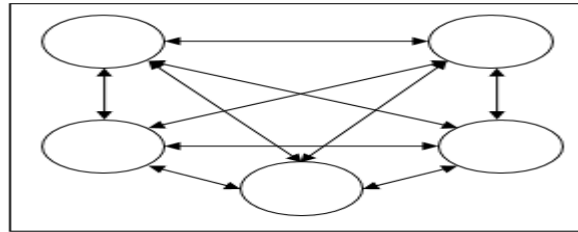


B. Multilayer feedforward network – The concept is of feedforward ANN having more than one weighted layer. As this network has one or more layers between the input and the output layer, it is called hidden layers.

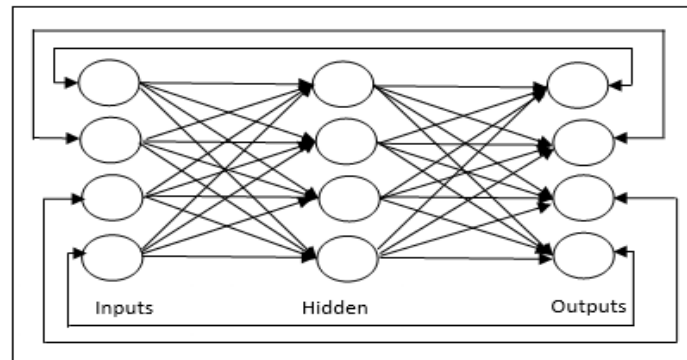


- **feedback network** has feedback paths, which means the signal can flow in both directions using loops. This makes it a non-linear dynamic system, which changes continuously until it reaches a state of equilibrium. It may be divided into the following types : **Recurrent networks**
 - They are feedback networks with closed loops. Following are the two types of recurrent networks.

A. Fully recurrent network – It is the simplest neural network architecture because all nodes are connected to all other nodes and each node works as both input and output.



B. Jordan network – It is a closed loop network in which the output will go to the input again as feedback as shown in the following diagram.



12. Adjustment of weight or Learning,

Learning, in artificial neural network, is the method of modifying the weights of connections between the neurons of a specified network. Learning in ANN can be classified into three categories namely supervised learning, unsupervised learning, and reinforcement learning

- **Supervised Learning** - As the name suggests, this type of learning is done under the supervision of a teacher. This learning process is dependent. During the training of ANN under supervised learning, the input vector is presented to the network, which will give an output vector. This output vector is compared with the desired output vector. An error signal is generated, if there is a difference between the actual output and the desired output vector. On the basis of this error signal, the weights are adjusted until the actual output is matched with the desired output.
- **Unsupervised learning** – is done without the supervision of a teacher. This learning process is independent. During the training of ANN under unsupervised learning, the input vectors of similar type are combined to form clusters. When a new input pattern is applied, then the neural network gives an output response indicating the class to which the input pattern belongs. There is no feedback from the environment as to what should be the desired output and if it is correct or incorrect. Hence, in this type of learning, the network itself must discover the patterns and features from the input data, and the relation for the input data over the output.

- **Reinforcement learning** - is used to reinforce or strengthen the network over some critic information. This learning process is similar to supervised learning, however we might have very less information. During the training of network under reinforcement learning, the network receives some feedback from the environment. This makes it somewhat similar to supervised learning. However, the feedback obtained here is evaluative not instructive, which means there is no teacher as in supervised learning. After receiving the feedback, the network performs adjustments of the weights to get better critic information in future.

13. Activation function

It may be defined as the extra force or effort applied over the input to obtain an exact output. In ANN, we can also apply activation functions over the input to get the exact output.

Can be

- Linear activation function - $f(x) = x$
- Sigmoid activation function
- Binary sigmoid

$$F(x) = \text{sigm}(x) = \frac{1}{1 + \exp(-x)}$$

- Bi-polar sigmoid

$$F(x) = \text{sigm}(x) = \frac{2}{1 + \exp(-x)} - 1 = \frac{1 - \exp(x)}{1 + \exp(x)}$$

- The net input to the output neuron is computed as
 - b is bias
 - X_i is input neuron
 - W_i is weight

$$y_{net} = b + \sum_{i=1}^n x_i w_i$$

6.6 Application of ANN

- **Aerospace** – Autopilot aircrafts, aircraft fault detection.
- **Military** – Weapon orientation and steering, target tracking, object discrimination, facial recognition, signal/image identification.
- **Electronics** – Code sequence prediction, IC chip layout, chip failure analysis, machine vision, voice synthesis.

- **Financial** – Real estate appraisal, loan advisor, mortgage screening, corporate bond rating, portfolio trading program, corporate financial analysis, currency value prediction, document readers, credit application evaluators.
- **Medical** – Cancer cell analysis, EEG and ECG analysis, prosthetic design, transplant time optimizer.
- **Speech** – Speech recognition, speech classification, text to speech conversion.
- **Telecommunications** – Image and data compression, automated information services, real-time spoken language translation.
- **Transportation** – Truck Brake system diagnosis, vehicle scheduling, routing systems.
- **Software** – Pattern Recognition in facial recognition, optical character recognition, etc.
- **Time Series Prediction** – ANNs are used to make predictions on stocks and natural calamities.
- **Signal Processing** – Neural networks can be trained to process an audio signal and filter it appropriately in the hearing aids.
- **Anomaly Detection** – As ANNs are expert at recognizing patterns, they can also be trained to generate an output when something unusual occurs that misfits the pattern.

6.7 Sample Questions

1. Which of the following is not the promise of artificial neural network?
 - A. It can explain result
 - B. It can survive the failure of some nodes
 - C. It has inherent parallelism
 - D. It can handle noise
2. Which of the following is the model used for learning?
 - A. Decision trees
 - B. Neural networks
 - C. Propositional and FOL rules
 - D. All of the mentioned
3. Neural Networks are complex _____ with many parameters.
 - A. Linear Functions
 - B. Nonlinear Functions
 - C. Discrete Functions
 - D. Exponential Functions
4. In which of the following learning the teacher returns reward and punishment to learner?
 - A. Active learning
 - B. Reinforcement learning
 - C. Supervised learning
 - D. Unsupervised learning
5. The network that involves backward links from output to the input and hidden layers is called
 - A. Self-organizing maps
 - B. Perceptron
 - C. Recurrent neural network
 - D. Multi layered perceptron

- A. 238
B. 76
C. 119
D. 123
- _____ is a decision support tool that uses a tree-like graph or model of decisions and their possible consequences, including chance event outcomes, resource costs, and utility.
- A. Decision tree
B. Graphs
C. Trees
D. Neural Networks
- In Unsupervised learning _____
- A. Specific output values are given
B. Specific output values are not given
C. No specific Inputs are given
D. Both inputs and outputs are given
- Which is true for neural networks?
- A. It has set of nodes and connections
B. Each node computes its weighted input
C. Node could be in excited state or non-excited state
D. All of the mentioned

Chapter Seven: Communicating, Perceiving, and Acting

Objectives

- Define NLP
- NLP components and its implication
- Pros and Cons of NLP
- Phases of NLP
- Difficulty of NLP
- Application of NLP
- Define Robotics and its application in details
- Define locomotion in robotics and discuss each types

7.1 Natural language processing

NLP stands for Natural Language Processing, which is a part of Computer Science, Human language, and Artificial Intelligence. It is the technology that is used by machines to understand, analyze, manipulate, and interpret human's languages. It helps developers to organize knowledge for performing tasks such as translation, automatic summarization, Named Entity Recognition (NER), speech recognition, relationship extraction, and topic segmentation.

● Advantages of NLP

- Helps users to ask questions about any subject and get a direct response within seconds.
- Helps computers to communicate with humans in their languages.
- It is very time efficient.
- Most of the companies use NLP to improve the efficiency of documentation processes, accuracy of documentation, and identify the information from large databases.

● Disadvantages of NLP

- NLP may not show context.
- NLP is unpredictable
- NLP may require more keystrokes.
- NLP is unable to adapt to the new domain, and it has a limited function that's why NLP is built for a single and specific task only.

➔ Components of NLP

1. Natural Language Understanding (NLU)

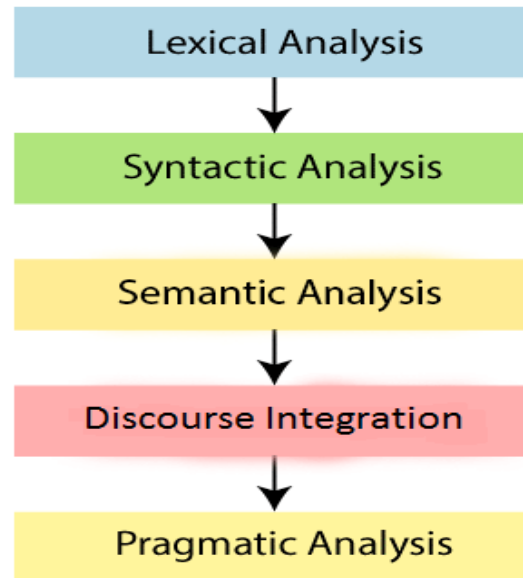
- Natural Language Understanding (NLU) helps the machine to understand and analyses human language by extracting the metadata from content such as concepts, entities, keywords, emotion, relations, and semantic roles.
- NLU mainly used in Business applications to understand the customer's problem in both spoken and written language.
- Tasks -
 - It is used to map the given input into useful representation.
 - It is used to analyze different aspects of the language.

2. Natural Language Generation (NLG)

- Natural Language Generation (NLG) acts as a translator that converts the computerized data into natural language representation. It mainly involves Text planning, Sentence planning, and Text Realization.
- The NLU is difficult than NLG.
- Difference between NLU and NLG

- NLU is the process of reading and interpreting language.
- NLG is the process of writing or generating language.
- NLG - It produces non-linguistic outputs from natural language inputs.
- NLG - It produces constructing natural language outputs from non-linguistic inputs.

➔ Phases of NLP



1. Lexical Analysis and Morphological

- The first phase of NLP is the Lexical Analysis. This phase scans the source code as a stream of characters and converts it into meaningful lexemes. It divides the whole text into paragraphs, sentences, and words.

2. Syntactic Analysis (Parsing)

- Syntactic Analysis is used to check grammar, word arrangements, and shows the relationship among the words.
- Example: Agra goes to the Poonam
- In the real world, Agra goes to the Poonam, does not make any sense, so this sentence is rejected by the Syntactic analyzer.

3. Semantic Analysis

- Semantic analysis is concerned with the meaning representation. It mainly focuses on the literal meaning of words, phrases, and sentences.

4. Discourse Integration

- Discourse Integration depends upon the sentences that proceeds it and also invokes the meaning of the sentences that follow it.

5. Pragmatic Analysis

- Pragmatic is the fifth and last phase of NLP. It helps you to discover the intended effect by applying a set of rules that characterize cooperative dialogues.
- For Example: "Open the door" is interpreted as a request instead of an order.

➔ Difficulty of NLP

- NLP is difficult because Ambiguity and Uncertainty exist in the language.
- Ambiguity
- There are the following three ambiguity -

1. Lexical Ambiguity

- Lexical Ambiguity exists in the presence of two or more possible meanings of the sentence within a single word. Example:
 - Manya is looking for a match.
 - In the above example, the word match refers to that either Manya is looking for a partner or Manya is looking for a match. (Cricket or other match)

2. Syntactic Ambiguity

- Syntactic Ambiguity exists in the presence of two or more possible meanings within the sentence.
- Example: I saw the girl with the binocular.
 - In the above example, did I have the binoculars? Or did the girl have the binoculars?

3. Referential Ambiguity

- Referential Ambiguity exists when you are referring to something using the pronoun.
- Example: Kiran went to Sunita. She said, "I am hungry."
- In the above sentence, you do not know that who is hungry, either Kiran or Sunita.

7.2 Applications of NLP

- Machine Translation -is used to translate text or speech from one natural language to another natural language.
- Question Answering focuses on building systems that automatically answer the questions asked by humans in a natural language.
- Spam detection is used to detect unwanted e-mails getting to a user's inbox.
- Sentiment Analysis is also known as opinion mining. It is used on the web to analyze the attitude, behavior, and emotional state of the sender.
- Spelling correction - Microsoft Corporation provides word processor software like MS-word, PowerPoint for the spelling correction.

- Speech Recognition - It is used for converting spoken words into text. It is used in applications, such as mobile, home automation, video recovery, dictating to Microsoft Word, voice biometrics, voice user interface, and so on.
- Chatbot - is one of the important applications of NLP. It is used by many companies to provide the customer's chat services.
- Information extraction - Information extraction is one of the most important applications of NLP.

7.3 Introduction to robotics

Robotics is a domain in artificial intelligence that deals with the study of creating intelligent and efficient robots. Robots are the artificial agents acting in real world environment. Robots are aimed at manipulating the objects by perceiving, picking, moving, modifying the physical properties of object, destroying it, or to have an effect thereby freeing manpower from doing repetitive functions without getting bored, distracted, or exhausted

- Robot locomotion
 - Locomotion is the mechanism that makes a robot capable of moving in its environment. There are various types of locomotion
 - Legged
 - Wheeled
 - Combination of Legged and Wheeled Locomotion
 - Tracked slip/skid
- Legged Locomotion
 - This type of locomotion consumes more power while demonstrating walk, jump, trot, hop, climb up or down, etc.
 - It requires more number of motors to accomplish a movement. It is suited for rough as well as smooth terrain where irregular or too smooth surface makes it consume more power for a wheeled locomotion. It is little difficult to implement because of stability issues.
 - It comes with the variety of one, two, four, and six legs. If a robot has multiple legs then leg coordination is necessary for locomotion.
 - The total number of possible **gaits** (a periodic sequence of lift and release events for each of the total legs) a robot can travel depends upon the number of its legs.
 - If a robot has k legs, then the number of possible events $N = (2k-1)!$.
 - In case of a two-legged robot ($k=2$), the number of possible events is $N = (2k-1)! = (2*2-1)! = 3! = 6$.
 - Hence there are six possible different events –

- Lifting the Left leg
- Releasing the Left leg
- Lifting the Right leg
- Releasing the Right leg
- Lifting both the legs together
- Releasing both the legs together
- In case of $k=6$ legs, there are 39916800 possible events. Hence the complexity of robots is directly proportional to the number of legs
- Wheeled Locomotion
 - It requires fewer numbers of motors to accomplish a movement. It is little easy to implement as there are less stability issues in case of more number of wheels. It is power efficient as compared to legged locomotion.
 - **Standard wheel** – Rotates around the wheel axle and around the contact
 - **Castor wheel** – Rotates around the wheel axle and the offset steering joint.
 - **Swedish 45° and Swedish 90° wheels** – Omni-wheel, rotates around the contact point, around the wheel axle, and around the rollers.
 - **Ball or spherical wheel** – Omnidirectional wheel, technically difficult to implement
- Slip/Skid Locomotion
 - In this type, the vehicles use tracks as in a tank. The robot is steered by moving the tracks with different speeds in the same or opposite direction. It offers stability because of large contact area of track and ground.

⇒ **Components of robotics**

- **Power Supply** – The robots are powered by batteries, solar power, hydraulic, or pneumatic power sources.
- **Actuators** – They convert energy into movement.
- **Electric motors (AC/DC)** – They are required for rotational movement.
- **Pneumatic Air Muscles** – They contract almost 40% when air is sucked in them.
- **Muscle Wires** – They contract by 5% when electric current is passed through them.
- **Piezo Motors and Ultrasonic Motors** – Best for industrial robots.
- **Sensors** – They provide knowledge of real time information on the task environment. Robots are equipped with vision sensors to be to compute the depth in the environment. A tactile sensor imitates the mechanical properties of touch receptors of human fingertips.

⇒ **Computer vision**

- This is a technology of AI with which the robots can see. The computer vision plays vital role in the domains of safety, security, health, access, and entertainment.
- Computer vision automatically extracts, analyzes, and comprehends useful information from a single image or an array of images. This process involves development of algorithms to accomplish automatic visual comprehension.
- Hardware of Computer Vision System - This involves –
 - Power supply
 - Image acquisition device such as camera
 - A processor
 - A software
 - A display device for monitoring the system
 - Accessories such as camera stands, cables, and connectors

⇒ **Tasks of Computer Vision**

- **OCR** – In the domain of computers, Optical Character Reader, a software to convert scanned documents into editable text, which accompanies a scanner.
- **Face Detection** – Many state-of-the-art cameras come with this feature, which enables to read the face and take the picture of that perfect expression. It is used to let a user access the software on correct match.
- **Object Recognition** – they are installed in supermarkets, cameras, high-end cars such as BMW, GM, and Volvo.
- **Estimating Position** – It is estimating position of an object with respect to camera as in position of tumor in human's body.

⇒ **Application of computer vision**

- **Industries** – Robots are used for handling material, cutting, welding, color coating, drilling, polishing, etc.
- **Military** – Autonomous robots can reach inaccessible and hazardous zones during war.
- **Medicine** – the robots are capable of carrying out hundreds of clinical tests simultaneously, rehabilitating permanently disabled people, and performing complex surgeries such as brain tumors.
- **Exploration** – the robot rock climbers used for space exploration, underwater drones used for ocean exploration are to name a few.
- **Entertainment** – Disney's engineers have created hundreds of robots for movie making.

7.4 Sample Questions

1. What is the complex system of structured message?

A. Languages

C. Signs

B. Words

D. Speech

2. What is a finite set of rules that specifies a language?

A. Signs

C. Grammar

B. Communication

D. Phrase

3. Semantic grammars are _____

A. Encode semantic information into a syntactic grammar

B. Decode semantic information into a syntactic grammar

C. Encode syntactic information into a semantic grammar

D. Decode syntactic information into a semantic grammar

4. Which of the following terms refers to the use of compressed gasses to drive (power) the robot device?

A. Pneumatic

C. Piezoelectric

B. Hydraulic

D. Photosensitive

5. With regard to the physics of power systems used operate robots, which statement or statements are most correct?

A. hydraulics involves the compression of liquids

B. hydraulics involves the compression of air

C. pneumatics involves the compression of air

D. chemical batteries produce AC power

6. Which of the following “laws” is Asimov’s first and most important law of robotics?

A. robot actions must never result in damage to the robot

B. robots must never take actions harmful to humans

C. robots must follow the directions given by humans

D. robots must make business a greater profit